

Win Probability and Decision Analysis: Fourth Downs, Two-Pointers, Timeouts

How a model turns score, clock and field position into a chance of winning, and how to use it to grade the biggest decisions a coach makes

Super Bowl LI, February 5, 2017. With 8:31 left in the third quarter, Tevin Coleman catches a 6-yard touchdown pass and Atlanta leads New England 28-3. The public win-probability model that powers this chapter put the Falcons' chance of winning at 99 percent. You know what happened next: the Patriots scored 31 unanswered points and won 34-28 in the first overtime in Super Bowl history.¹ So was the model wrong? Or was this the one game in a hundred it had warned about? And when coaches face a fourth down or a two-point try, how do the same models tell them what to do? This chapter builds a win-probability model from scratch, checks whether it tells the truth, and uses it on the decisions that fans argue about most: going for it on fourth down, going for two, and spending timeouts.

You'll be able to:

- Explain what a win-probability model estimates, fit a simple one yourself, and say how it differs from nflverse's model and from the betting line.
- Read a win-probability chart, compute WPA for any play, and use leverage to say which moments of a game matter most.
- Build a fourth-down decision tree (conversion probability times the win probability of each outcome, against the field goal and the punt) and compute the break-even conversion rate.
- Work the down-14 two-point decision by hand and say when going for two early is right.
- Critique a model's advice: check its calibration, find where it breaks, and say how uncertain its recommendations are.
- Measure how coaches' decisions have changed, league-wide and team by team.

You'll need

- **Expected Points, EPA, and Success Rate:** **expected points** (EP, the average value of the next score from a down, distance and yard line), **EPA** (EP after a play minus EP before it), and the habit of checking a model against the raw data. This chapter uses nflverse's EP as an input.
- **The Clock, the Two-Minute Drill, and Fourth Down:** the **go-for-it decision**, the fourth-down rules of thumb, the **two-point chart**, and what **timeouts** buy at the end of a half. That chapter showed *what* teams decide; this one does the math.
- **Working with nflverse Data in Python:** loading play-by-play data and the `yardline_100` column (yards from the opponent's goal line).

The data this chapter uses

As everywhere in Part 13, the code is part of the text. The cell below loads ten regular seasons of nflverse play-by-play (2016-2025), the same window as Expected Points, EPA, and Success Rate, and adds two columns every model in this chapter needs: how the game ended for the team with the ball, and how many minutes were left.²

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch, Circle
```

¹Super Bowl LI, February 5, 2017, NRG Stadium, Houston: New England 34, Atlanta 28 (OT), the first Super Bowl to go to overtime and the largest comeback in Super Bowl history (25 points); Atlanta led 28-3 after Tevin Coleman's touchdown with 8:31 left in the third quarter: https://en.wikipedia.org/wiki/Super_Bowl_LI. Win probabilities are nflverse's `home_wp` for game 2016_21_NE_ATL (Atlanta was the designated home team).

²nflverse play-by-play loaded with `nflreadpy` (<https://github.com/nflverse/nflreadpy>) and cached in `data/cache/`; regular seasons 2016-2025 (plus the two playoff games named in the text); data retrieved October 2026. nflverse revises past seasons occasionally, so re-running the code can move the last decimal place. Columns are described in the play-by-play dictionary: https://nflreadr.nflverse.com/articles/dictionary_pbp.html.

```

from matplotlib.colors import LinearSegmentedColormap
from IPython.display import Markdown
from gridiron import style
from gridiron.data import pbp

style.apply()
SEASONS = range(2016, 2026) # ten completed seasons, 2016-2025
COLS = ["season", "season_type", "week", "game_id", "play_id", "drive", "qtr",
        "quarter_seconds_remaining", "half_seconds_remaining", "game_seconds_remaining",
        "posteam", "defteam", "home_team", "away_team", "down", "ydstogo", "yardline_100",
        "goal_to_go", "play_type", "yards_gained", "score_differential", "result",
        "ep", "epa", "wp", "vegas_wp", "wpa", "home_wp", "home_wp_post", "vegas_home_wp",
        "posteam_timeouts_remaining", "defteam_timeouts_remaining", "timeout", "timeout_team",
        "fourth_down_converted", "field_goal_result", "kick_distance", "qb_kneel", "qb_spike",
        "two_point_attempt", "two_point_conv_result", "extra_point_attempt",
        "extra_point_result", "fixed_drive_result", "desc"]
raw = pbp(SEASONS, columns=COLS)
reg = raw[(raw.season_type == "REG") & (raw.qtr <= 4)].copy() # regulation, regular season

# How the game ended, from the point of view of the team with the ball.
final = np.where(reg.posteam == reg.home_team, reg.result, -reg.result) # its final margin
reg["final"] = final
reg["win"] = np.select([final > 0, final < 0], [1.0, 0.0], 0.5) # a tie counts half
reg["minutes"] = reg.game_seconds_remaining / 60

# "Snaps": every play with a down, run from scrimmage or kicked.
snaps = reg[reg.down.notna() & reg.ep.notna()
            & reg.play_type.isin(["pass", "run", "punt", "field_goal"])].copy()

def md_table(df, digits=1, widths=None):
    """Show a small DataFrame as a Markdown table (presentation plumbing, not football).
    `widths` sets relative column widths (pandoc reads them from the dashes)."""
    out = df.copy()
    for c in out.select_dtypes("number"):
        whole = (out[c].dropna() % 1 == 0).all() # counts print without decimals
        out[c] = out[c].map(lambda v: f"{v:.{0 if whole else digits}f}")
    widths = widths or [3] * len(df.columns)
    align = [("-" * w + ":") if pd.api.types.is_numeric_dtype(df[c]) else (":" + "-" * w)
             for c, w in zip(df, widths)]
    rows = [list(out.columns), align] + out.astype(str).values.tolist()
    return Markdown("\n".join("| " + " | ".join(map(str, r)) + " |" for r in rows))

def pct(v, d=0):
    return f"{100 * v:.{d}f}%"

```

Two choices matter. **Regulation only**: overtime has its own rules (both teams get a possession since 2025 in the regular season, and a tie is possible), so its plays are left out of the model. **A tie counts as half a win**, the usual convention, so that the target is always a number between 0 and 1.

From points to wins

Expected Points, EPA, and Success Rate priced every situation in points. Points are the right currency for most of a game, but they are not what a coach is trying to maximise. He is trying to win. Usually the two agree. Sometimes they don't:

- Leading by 4 with two minutes left, a field goal is worth the same 3 points as one in the first quarter, but it changes the game far more: the opponent now needs a touchdown *and* has less time.
- Trailing by 10 with three minutes left, a field goal is worth 3 points but may be almost worthless: you still need a touchdown and an onside kick, and you used up time getting the three.
- Trailing by 17 in the fourth quarter, a 6-yard completion on third-and-10 is a bad play by EPA and barely matters by any measure, because the game is all but over.

Win probability (WP) is the score-aware cousin of expected points: the chance that a team wins the game from here, given the score, the time left, field position, down and distance and, in fuller models, timeouts and how good the two teams are. “Atlanta 99%” means: of all the teams that have been in a situation like Atlanta’s, about 99 in 100 went on to win. You have seen the broadcast version since How to Watch a Broadcast, the bar under the scorebug. This chapter opens it up.

! Why it exists

Coaches make dozens of decisions a game whose costs and benefits arrive in different currencies: yards, points, possessions, seconds. Expected points converts the first three into one unit, but it ignores the score and treats a minute of clock in the first quarter like a minute in the fourth. Win probability converts *everything*, the clock and the score included, into the only unit that ends up in the standings. That is what makes it the natural yardstick for late-game decisions, and the right final judge even for early ones.

A football game is a random walk

Here is the idea that makes win probability tractable. Write down a team’s lead plus the expected points of the possession it has right now. Call that its **effective lead**: leading by 3 with first-and-10 at the opponent’s 20 (worth about +4.6 points) is an effective lead of about 7.6. Now ask how far the *final* margin ends up from the effective lead. If football scoring were a random walk, a series of small, independent steps up and down, the spread of that gap would grow with the square root of the time left, the same way the spread of a coin-flip tally grows with the square root of the number of flips. The cell below checks.

```
snaps["eff_lead"] = snaps.score_differential + snaps.ep          # lead + value of this possession
snaps["gap"] = snaps.final - snaps.eff_lead                    # what happened from here on
by_min = (snaps.assign(m=np.ceil(snaps.minutes).clip(1, 60))
          .groupby("m").gap.agg(["std", "mean", "size"]))
per_root_min = (by_min["std"] / np.sqrt(by_min.index)).loc[5:60]
print(f"Spread of (final margin - effective lead) per square-root minute, 5-60 min left: "
      f"{per_root_min.min():.2f} to {per_root_min.max():.2f}")
print(f"Spread with a full game left: {by_min.loc[60, 'std']:.1f} points")
```

```
Spread of (final margin - effective lead) per square-root minute, 5-60 min left: 1.73 to 1.99
Spread with a full game left: 14.2 points
```

```
fig, ax = plt.subplots(figsize=(6.5, 3.4))
m = by_min.index.to_numpy()
ax.plot(m, 1.85 * np.sqrt(m), color=style.SERIES[0], lw=2, label="1.85 × √(minutes left)")
ax.scatter(m, by_min["std"], s=14, color=style.SERIES[1], zorder=3,
           edgecolor=style.SURFACE, linewidth=0.6, label="Actual spread")
ax.set_xlim(61, 0)
ax.set_ylim(0, 16)
ax.set_xticks([60, 45, 30, 15, 0])
ax.set_xlabel("Minutes left in regulation")
ax.set_ylabel("Points (standard deviation)")
ax.set_title("The uncertainty left in a game shrinks like √time")
ax.legend(loc="upper right")
plt.show()
```

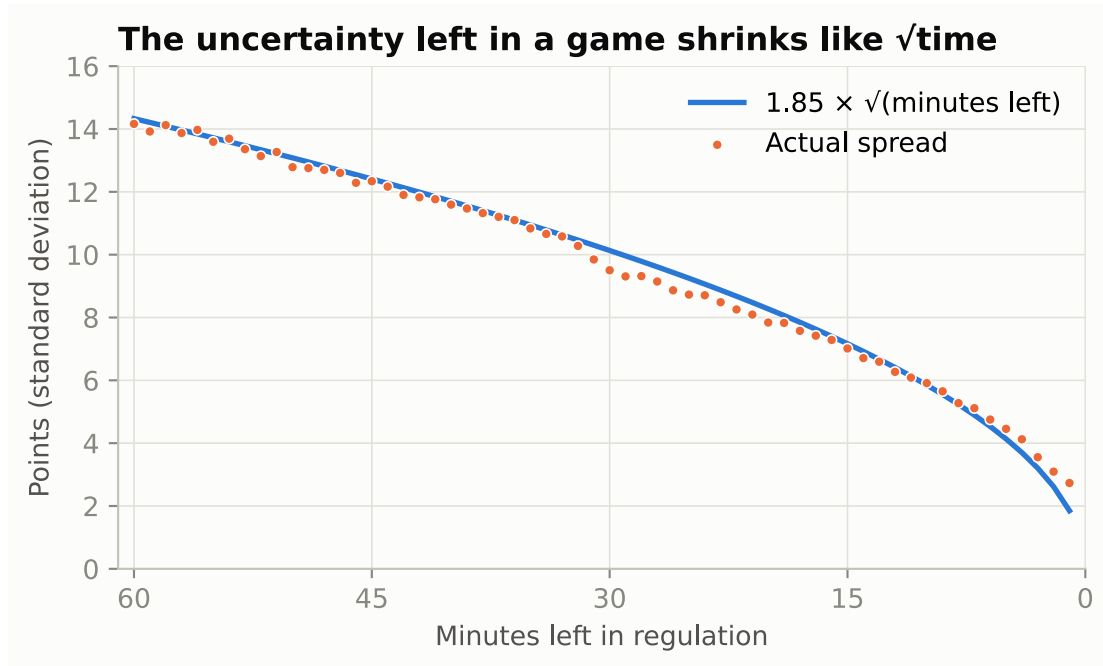


Figure 1: How far the final margin lands from the effective lead (current lead plus the expected points of the possession), by minutes left, 2016-2025 regular seasons. Dots: the actual standard deviation at each minute. Line: a constant 1.85 points per square-root minute. The fit is close almost all the way down; in the last few minutes the dots sit above the line because points come in lumps of 3 and 7 and the clock can be run out, which a smooth random walk doesn't know.

The square-root law holds remarkably well: from the opening kickoff to five minutes left, the spread is a nearly constant 1.83 points per square-root minute. With a whole game left it is about 14 points, which matches a famous number: in 1991 the statistician Hal Stern showed that NFL final margins scatter around the betting line with a standard deviation of about 14 points, close enough to a normal (bell-shaped) distribution that you can turn a point spread into a win probability with a table from a statistics textbook. Three years later he extended the idea within a game, treating the score as a **Brownian motion**, the continuous version of a random walk.³ The argument fits in one line: if the rest of the game adds a normally distributed amount to the effective lead, with mean zero and standard deviation $\sigma\sqrt{t}$, then the team wins whenever that amount doesn't wipe out its lead, and

$$WP = \Phi\left(\frac{\text{lead} + EP}{\sigma\sqrt{t} + \tau}\right)$$

where Φ is the normal cumulative distribution (the share of a bell curve below a point), t is the minutes left, σ is the spread per square-root minute, and τ is a small cushion that stops the formula from becoming infinitely sure in the final seconds. That is a complete win probability model with two numbers to fit. The cell fits them on 2016-2024 and holds 2025 back as a test.

```
def Phi(z):
    """Normal CDF. NumPy has no erf, so this is the classic Abramowitz-Stegun 7.1.26
    approximation (error below 2e-7), written with NumPy so it runs on whole columns."""
    z = np.asarray(z, dtype=float)
    x = np.abs(z) / np.sqrt(2)
    t = 1 / (1 + 0.3275911 * x)
    poly = t * (0.254829592 + t * (-0.284496736 + t * (1.421413741
        + t * (-1.453152027 + t * 1.061405429))))
    return 0.5 * (1 + np.sign(z) * (1 - poly * np.exp(-x * x)))

def log_loss(p, y):
```

³Hal Stern, "On the Probability of Winning a Football Game," *The American Statistician* 45, no. 3 (1991): 179-183 (NFL margins around the point spread approximately normal with standard deviation about 13.86), <https://doi.org/10.1080/00031305.1991.10475798>; Hal S. Stern, "A Brownian Motion Model for the Progress of Sports Scores," *Journal of the American Statistical Association* 89, no. 427 (1994): 1128-1134, <https://doi.org/10.1080/01621459.1994.10476851>.

```

p = np.clip(p, 1e-4, 1 - 1e-4)
return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

train, test = snaps[snaps.season <= 2024], snaps[snaps.season == 2025].copy()
grid = [(log_loss(Phi(train.eff_lead / (s * np.sqrt(train.minutes + tau))), train.win), s, tau)
         for s in np.arange(1.70, 2.21, 0.025) for tau in (0.5, 1, 1.5, 2, 2.5, 3, 4)]
_, SIG, TAU = min(grid)

def wp_model(lead, ep, minutes):
    """Win probability for the team with the ball: lead and EP in points, minutes left."""
    return Phi((np.asarray(lead) + np.asarray(ep))
               / (SIG * np.sqrt(np.maximum(minutes, 0) + TAU)))

test["my_wp"] = wp_model(test.score_differential, test.ep, test.minutes)
print(f"Fitted: sigma = {SIG:.3f} points per root-minute, tau = {TAU} minutes")

```

```
Fitted: sigma = 1.925 points per root-minute, tau = 3 minutes
```

The fitted σ is 1.92, a little above the raw spread, because a model judged by how well it predicts wins prefers to be slightly less sure of itself than the raw spread suggests (a theme that returns below). With $\tau = 3$ minutes, here is what the model says.

```

fig, ax = plt.subplots(figsize=(6.5, 3.6))
eff = np.linspace(-17, 17, 200)
for k, mins in enumerate([55, 30, 15, 5, 2]):
    ax.plot(eff, wp_model(eff, 0, mins), color=style.SERIES[k], lw=2,
            label=f"{mins} min left")
ax.axhline(0.5, color=style.BASELINE, lw=1)
ax.axvline(0, color=style.BASELINE, lw=1)
ax.set_xlim(-17, 17)
ax.set_ylim(0, 1)
ax.set_xticks(range(-14, 15, 7))
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
ax.set_xlabel("Effective lead (points): score margin + EP of the possession")
ax.set_ylabel("Win probability")
ax.set_title("Same lead, different clock")
ax.legend(loc="upper left", fontsize=8)
plt.show()

```

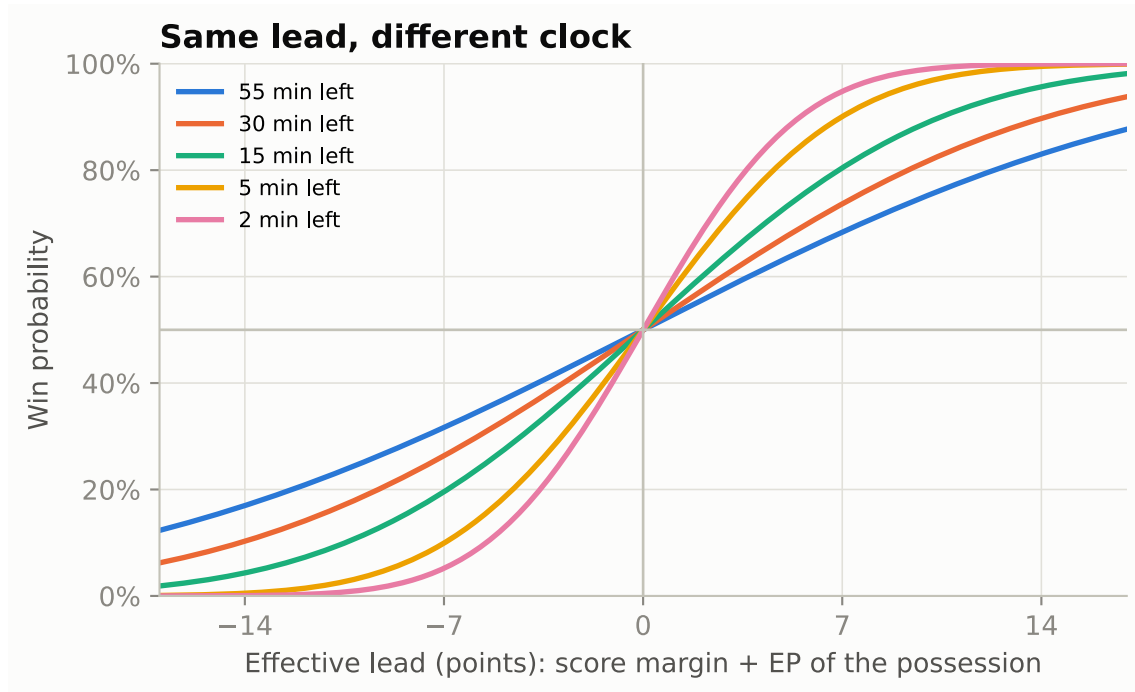


Figure 2: The random-walk model's win probability for the team with the ball, by effective lead (lead plus the expected points of its possession), at five points in a game. Early, a 7-point effective lead is worth about two chances in three; with two minutes left the same lead is worth about 95%. The curves get steeper as time runs out because there is less game left to undo the lead.

Read the chart the way a coach feels a game. With 55 minutes left, even a 14-point effective lead is only about 83%: there is a lot of football left. With two minutes left, a 7-point effective lead is 95%. And every curve passes through 50% at an effective lead of zero, which is a reminder that the model knows nothing about *which* teams are playing.

How good is two numbers?

The fair test is the 2025 season, which the model never saw. Three models score every 2025 regulation snap: ours; nflverse's `wp`, a gradient-boosted tree model (XGBoost) with about a dozen inputs; and nflverse's `vegas_wp`, the same kind of model told the pregame point spread, so it knows which team was expected to win.⁴ Two scores are used. The **log loss** (from 13-02: the average of $-\log$ of the probability given to what actually happened) punishes confident misses hardest. The **Brier score** is the average squared gap between the probability and the outcome (1 for a win, 0 for a loss); lower is better, and a model that always said 50% would score 0.25. Both are standard ways to grade probability forecasts, and the Brier score is how Film Study and Charting will grade your own predictions.

```
t25 = test[test.wp.notna() & test.vegas_wp.notna()]
md_table(pd.DataFrame({
    "model (2025 regular season)": ["Random walk (this chapter, 2 numbers)",
                                    "nflverse wp (XGBoost)", "nflverse vegas_wp (+ point spread)",
                                    "Always 50%"],
    "log loss": [log_loss(t25.my_wp, t25.win), log_loss(t25.wp, t25.win),
                 log_loss(t25.vegas_wp, t25.win), log_loss(np.full(len(t25), .5), t25.win)],
    "Brier score": [((t25.my_wp - t25.win) ** 2).mean(), ((t25.wp - t25.win) ** 2).mean(),
                    ((t25.vegas_wp - t25.win) ** 2).mean(), 0.25]
})).to_markdown(digits=4)
```

model (2025 regular season)	log loss	Brier score
Random walk (this chapter, 2 numbers)	0.5057	0.1704
nflverse wp (XGBoost)	0.5037	0.1699

⁴Ben Baldwin, "nflfastR EP, WP, and CP models," Open Source Football, 2020 (the XGBoost win-probability model, its inputs, and the separate spread-adjusted version): <https://www.opensourcefootball.com/posts/2020-09-28-nflfast-ep-wp-and-cp-models/>.

model (2025 regular season)	log loss	Brier score
nflverse vegas_wp (+ point spread)	0.4761	0.1579
Always 50%	0.6931	0.2500

Two numbers do almost exactly as well as the machine-learning model. That is less a compliment to our model than a lesson about what win probability *is*: most of it is the score, the clock and the value of the possession, and a random walk captures exactly those. What moves the needle is **information about the teams**: the point-spread version wins clearly, because a 4-point lead for the better team is worth more than the same lead for the worse one. Keep that in mind on Sunday. A broadcast bar built without the spread is describing an average team in this team's shoes.

Common misconception

“Win probability is the betting odds.” It isn't, unless the model was given the betting line. nflverse publishes both: `wp` treats the teams as interchangeable (plus home field), `vegas_wp` starts from the pregame spread and lets that information fade as the game goes on. Broadcast models differ, and most don't say which they use. When two win-probability numbers for the same moment disagree by 10 points early in a game, the likeliest reason is that one of them knows who is favoured.

What the real models add

Our model is a teaching model, and it is worth being precise about what it leaves out, because every item on this list reappears when we use it for decisions:

- **Timeouts.** A trailing team with three timeouts can stop the clock three times; ours can't tell.
- **Who gets the ball at halftime.** The team receiving the second-half kickoff has an extra possession in hand, worth a few percentage points in a close game.
- **Lumpy scoring.** Points come in 3s, 7s and 8s. Down 4 late, a field goal is nearly worthless; down 3, it ties. A smooth random walk can't see that difference, which is the main reason its spread misses in the last few minutes (Figure 1).
- **Home field and team strength**, which `vegas_wp` handles through the spread.

The history of the field is the history of adding those things. Brian Burke's win-probability model on his Advanced NFL Stats site (from 2008 or so) used historical situations smoothed by hand; Dennis Lock and Dan Nettleton fitted a random forest in 2014; the nflscrapR project (2017) used a generalised additive model, and nflfastR (2020) the XGBoost model in today's nflverse data, with inputs for the score, the clock, the spread, timeouts, field position, down and distance and the second-half kickoff.⁵ ESPN, Next Gen Stats and the networks run their own proprietary versions. They all agree on the big picture and differ in the details, which matters for close decisions.

Is the model honest? Calibration

A win-probability model makes a promise: when it says 70%, the team should win about 70% of the time. Checking that promise is called checking its **calibration** (a forecast is calibrated when the things it calls 70% likely happen about 70% of the time; the term is taught in Film Study and Charting, where you'll calibrate your own predictions). Sort every 2025 snap by its forecast, cut into bins, and compare each bin's average forecast with how often those teams actually won.

```
bins = np.linspace(0, 1, 21)
fig, ax = plt.subplots(figsize=(5.2, 4.6))
ax.plot([0, 1], [0, 1], color=style.BASELINE, lw=1, zorder=1)
for k, (col, name) in enumerate([("my_wp", "Random walk (this chapter)"), ("wp", "nflverse wp")]):
    cut = pd.cut(t25[col], bins, include_lowest=True)
    cal = t25.groupby(cut, observed=True).agg(f=(col, "mean"), won=("win", "mean"))
    ax.plot(cal.f, cal.won, marker="o", ms=4.5, lw=1.2, color=style.SERIES[k], label=name)
ax.set_xlim(0, 1)
```

⁵Brian Burke's win-probability work at Advanced NFL Stats (founded 2007): ESPN Press Room bio, <https://espnpressroom.com/us/bios/brian-burke/>; Dennis Lock and Dan Nettleton, “Using random forests to estimate win probability before each play of an NFL game,” *Journal of Quantitative Analysis in Sports* 10, no. 2 (2014): 197-205, <https://doi.org/10.1515/jqas-2013-0100>; Ronald Yurko, Samuel Ventura and Maksim Horowitz, “nflWAR,” *Journal of Quantitative Analysis in Sports* 15, no. 3 (2019) (nflscrapR's generalised additive WP model), <https://doi.org/10.1515/jqas-2018-0010>.

```

ax.set_ylim(0, 1)
fmt = plt.FuncFormatter(lambda v, _: f"{v:.0%}")
ax.xaxis.set_major_formatter(fmt)
ax.yaxis.set_major_formatter(fmt)
ax.set_xlabel("Forecast win probability")
ax.set_ylabel("Share that actually won")
ax.set_title("When the model says 70%, do teams win 70%?")
ax.legend(loc="upper left", fontsize=8)
plt.show()

```

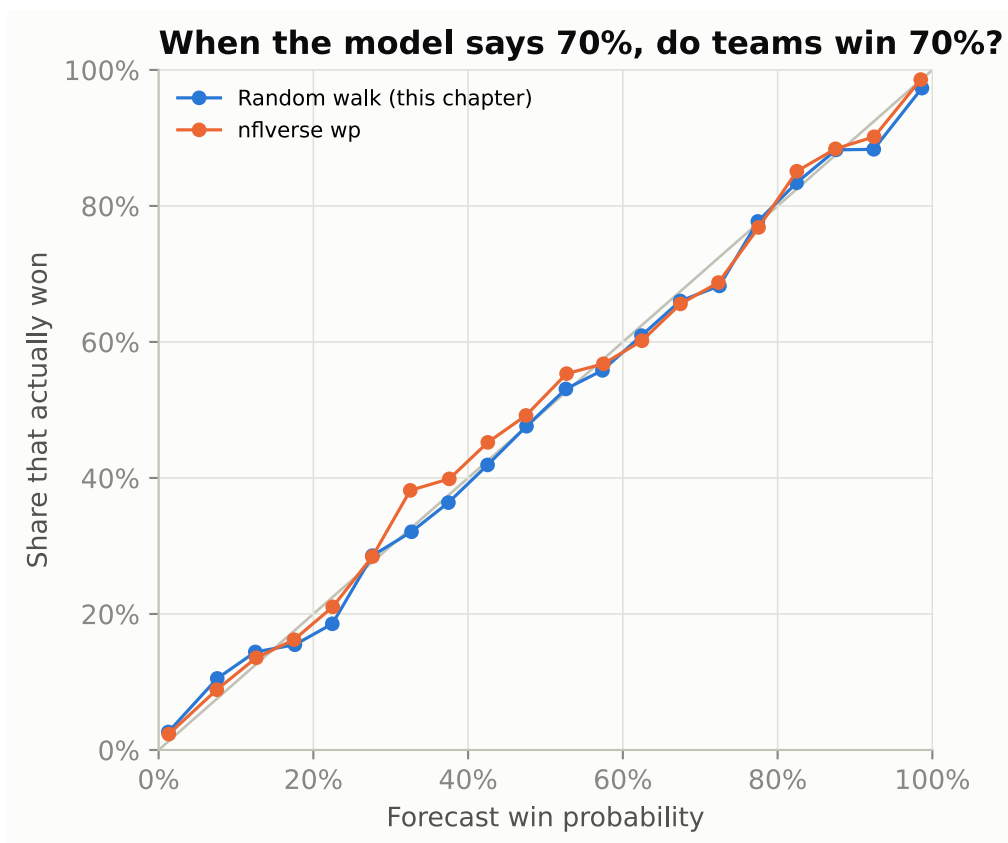


Figure 3: Calibration on the 2025 regular season (every regulation snap). Each dot is a bin of plays with similar forecasts: its position across is the average forecast, its height the share of those teams that actually won. A perfectly calibrated model puts every dot on the diagonal. Both models are close; both are a little overconfident at the extremes, where long-shot comebacks happened somewhat more often than forecast (the numbers are printed below the chart).

```

lo_t = t25[t25.wp < 0.05]
hi_t = t25[t25.wp > 0.95]
print(f"nflverse wp under 5%: average forecast {lo_t.wp.mean():.1%}, actual wins "
      f"{lo_t.win.mean():.1%} ({len(lo_t):,} snaps); over 95%: forecast {hi_t.wp.mean():.1%}, "
      f"actual {hi_t.win.mean():.1%}")

```

```

nflverse wp under 5%: average forecast 1.3%, actual wins 2.3% (2,704 snaps); over 95%: forecast 98.5%,
actual 98.6%

```

Both models sit close to the diagonal through the middle, which is where most decisions live. At the edges both are a touch overconfident: comebacks from long odds happened somewhat more often in 2025 than the models allowed. That is a common finding, and it has a plausible football reason: a team far behind changes how it plays (it passes on every down and goes for it on fourth down), and the team ahead often turns conservative, while the models learned from all situations, most of them played straight.

Now return to the opening question. A model that says 99% and is right should *still* see the 1% happen, about once in every hundred such games. Over a decade of NFL seasons there are hundreds of games in which somebody reached 99%, so a few historic collapses are not evidence that the model is broken; a *flood* of them would be. One game cannot tell you whether a probability was right. Many games can, and that is what a calibration chart is for.

🔥 Common misconception

“The model said 99% and they lost, so the model was wrong.” A probability is a claim about many games, not one. The Falcons’ 99% means that teams in their position lose about one time in a hundred, and in Super Bowl LI they did. Judge a forecaster by its calibration over hundreds of games, the way you would judge a weather forecaster by whether it rains on about 30% of the days it says “30%.”

Super Bowl LI, play by play

A win-probability model gives every play a score. **WPA (win probability added)** is the change in a team’s win probability from just before a play to just after it: the WP version of EPA. Add up a team’s WPA over a game and you get its final result (1 for a win) minus its starting win probability, so WPA tells you how the game was won, play by play. Here is Super Bowl LI through nflverse’s model.

```
sb = raw[raw.game_id == "2016_21_NE_ATL"].copy()          # Atlanta was the designated home team
sb["elapsed"] = (sb.qtr - 1) * 15 + 15 - sb.quarter_seconds_remaining / 60
sb = sb[sb.home_wp.notna()]
sb["ne_wpa"] = -(sb.home_wp_post - sb.home_wp)           # New England's WPA on each play
peak = sb.loc[sb.home_wp.idxmax()]
print(f"Atlanta's peak WP: {peak.home_wp:.1%} (vegas_wp version {sb.vegas_home_wp.max():.1%}), "
      f"{int(peak.qtr)}Q {peak.desc[:6]}")
```

Atlanta's peak WP: 99.4% (vegas_wp version 98.7%), 3Q (4:49)

```
fig, ax = plt.subplots(figsize=(6.6, 3.9))
ax.plot(sb.elapsed, sb.home_wp, color=style.SERIES[1], lw=2, label="Atlanta WP (nflverse wp)",
        drawstyle="steps-post")
ax.plot(sb.elapsed, sb.vegas_home_wp, color=style.SERIES[1], lw=1, ls=(0, (3, 2)),
        alpha=0.8, label="Atlanta WP, with point spread", drawstyle="steps-post")
ax.axhline(0.5, color=style.BASELINE, lw=1)
for q in (15, 30, 45, 60):
    ax.axvline(q, color=style.GRID, lw=0.8, zorder=0)
ax.text(60.4, 0.03, "OT", fontsize=7.5, color=style.INK_2)
NOTES = [ # (elapsed minute, wp to point at, text, text x, text y)
    (36.5, 0.990, "28-3 (8:31, Q3)", 37.5, 0.75),
    (51.5, 0.994, "Hightower strip-sack\n(8:31, Q4)", 38.0, 0.52),
    (56.1, 0.960, "Sack + holding push ATL\nout of FG range (3:56)", 38.0, 0.30),
    (57.6, 0.924, "Edelman catch\n(2:28)", 49.5, 0.11),
    (59.05, 0.606, "28-28 after\nNTD + 2 (0:57)", 61.2, 0.86),
]
for x0, y0, text, tx, ty in NOTES:
    ax.annotate(text, (x0, y0), (tx, ty), fontsize=7, color=style.INK, ha="left", va="center",
                bbox=dict(boxstyle="round,pad=0.25", fc=style.SURFACE, ec=style.BASELINE, lw=0.5),
                arrowprops=dict(arrowstyle="-", color=style.INK_2, lw=0.7, shrinkA=0, shrinkB=2))
ax.set_xlim(0, 70)
ax.set_ylim(0, 1.02)
ax.set_xticks([0, 15, 30, 45, 60], ["Kickoff", "Q2", "Q3", "Q4", "End of\nregulation"])
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
```

```
ax.set_ylabel("Atlanta's win probability")
ax.set_title("Super Bowl LI: Patriots 34, Falcons 28 (OT)")
ax.legend(loc="lower left", fontsize=7.5)
plt.show()
```

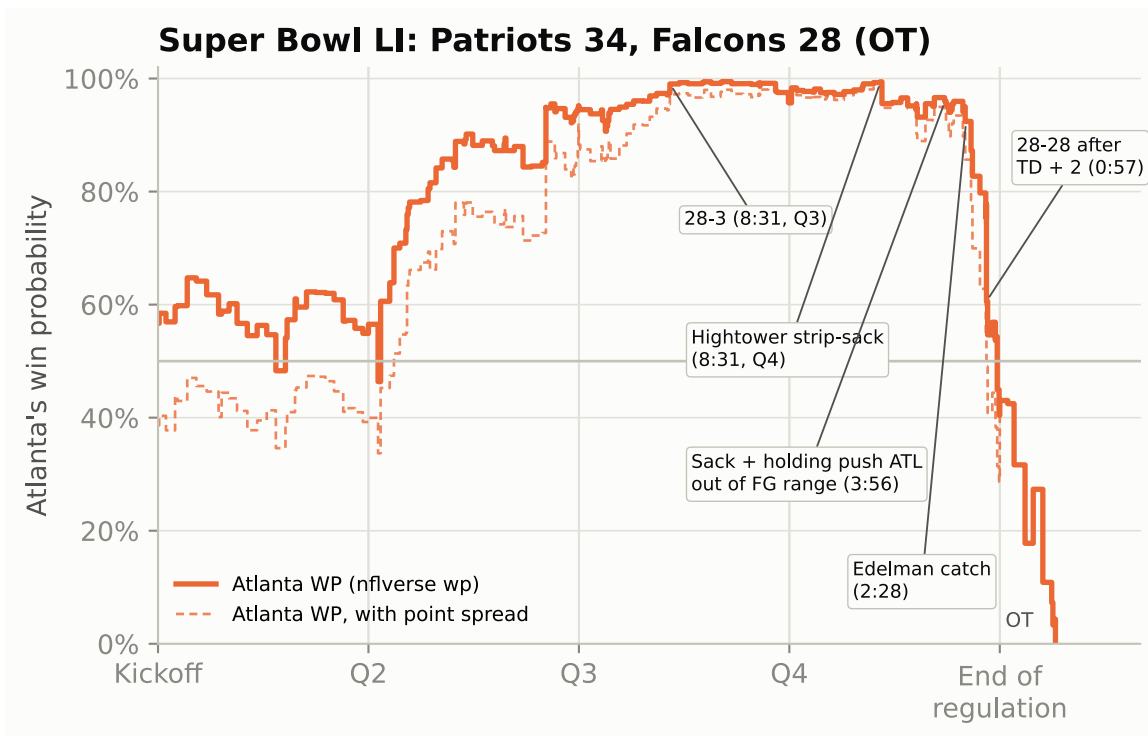


Figure 4: Super Bowl LI (February 5, 2017): Atlanta's win probability on every snap, from nflverse's model (solid) and its point-spread version (dashed; New England was favoured by about three points). The Falcons were above 95% for nearly 50 minutes of game time and peaked above 99% after Tevin Coleman's touchdown made it 28-3. The bar barely moved during New England's first three scoring drives; it collapsed only in the last four minutes of regulation, when every play carried high leverage.

The biggest single swings tell the story more compactly than any highlight package:

```
def short_desc(d, n=60):
    """Play text without the clock, formation tags and tacklers' names, cut at a word."""
    d = re.sub(r"\\(\\d+:\\d+\\)\\((Shotgun|No Huddle)\\)\\([\\^]*\\d+-[A-Z][\\^]*\\)\\([\\^\\]]*\\)", "", d)
    d = re.sub(r"\\s+", " ", d).strip()
    return d if len(d) <= n else d[:n].rsplit(" ", 1)[0] + " ..."

top = sb.reindex(sb.ne_wpa.abs().sort_values(ascending=False).index).head(8)
md_table(top.assign(
    when=lambda t: t.qtr.map({1: "Q1", 2: "Q2", 3: "Q3", 4: "Q4", 5: "OT"}) + " "
    + (t.quarter_seconds_remaining // 60).astype(int).astype(str) + ":"
    + (t.quarter_seconds_remaining % 60).astype(int).astype(str).str.zfill(2),
    play=lambda t: t.posteam + ": " + t.desc.map(short_desc),
    NE_WPA=lambda t: 100 * t.ne_wpa)[["when", "play", "NE_WPA"]
    .rename(columns={"NE_WPA": "NE WPA (pts of %)"}, widths=[8, 50, 10])
```

when	play	NE WPA (pts of %)
Q4 0:57	NE: TWO-POINT CONVERSION ATTEMPT. 12-T.Brady pass to ...	17.2

when	play	NE WPA (pts of %)
OT 12:37	NE: 12-T.Brady pass short left to 11-J.Edelman to ATL 25 for 15 ...	16.5
Q2 14:19	NE: 29-L.Blount left tackle to ATL 29 for 4 yards . FUMBLES , ...	-14.2
OT 13:59	NE: 12-T.Brady pass short left to 15-C.Hogan to ATL 37 for 18 ...	13.9
OT 14:26	NE: 12-T.Brady pass short right to 80-D.Amendola ran ob at NE ...	10.8
Q2 15:00	NE: 12-T.Brady pass deep right to 11-J.Edelman to ATL 33 for 27 ...	10.1
Q2 2:36	NE: 12-T.Brady pass short left intended for 80-D.Amendola ...	-9.8
OT 13:12	NE: 12-T.Brady pass short left to 28-J.White to ATL 40 for -3 ...	-9.6

Three things are worth noticing. First, **the scoring plays of the comeback are not on the list**. New England's touchdowns early in the fourth quarter barely moved the needle, because down 16 or 8 with plenty of time left still meant needing everything to go right. Second, **the biggest swings are the tying two-point conversion and the overtime plays**: from down 2 with under a minute left, two points turned a probable loss into a coin flip, bigger than the touchdown that preceded it, and in overtime each completion moved New England most of the way to the end zone in a game that the next score would end. Third, **turnovers in a close game are expensive at any time**. LeGarrette Blount's fumble and Robert Alford's pick-six came in the second quarter, but in a scoreless or one-score game they moved win probability about as much as anything in the fourth. Notice too what the model did *not* do: Atlanta's 99% never "changed its mind" at a single moment. It decayed through a long series of plays that each moved it a little, until the final four minutes, when the leverage of every snap exploded.

i Film room: Falcons 2016, Super Bowl LI

The play that analysts point to came with 4:40 left. Atlanta, leading 28-20, had first-and-10 at the New England 22, already in field-goal range. A field goal would have made it 31-20, an 11-point lead: two scores with under four minutes left. Instead the Falcons threw on second down; Matt Ryan was sacked by Trey Flowers for a 12-yard loss, a holding penalty on the next snap pushed them back another 10 yards, and they punted.⁶

```
lead_d = reg[(reg.qtr == 4) & reg.game_seconds_remaining.between(180, 240) & (reg.down == 1)
             & (reg.ydstogo == 10) & reg.yardline_100.between(60, 80)]
first_snap = lead_d.drop_duplicates("game_id")
down11 = first_snap[first_snap.score_differential.between(-11, -9)]
down8 = first_snap[first_snap.score_differential == -8]
print(f"Teams starting a drive 9-11 points down with 3-4 minutes left won "
      f"{down11.win.mean():.1%} of {len(down11)}; 8 points down: {down8.win.mean():.1%} of {len(down8)}")
```

Teams starting a drive 9-11 points down with 3-4 minutes left won 1.3% of 75; 8 points down: 10.5% of 19

What to notice: the gap between "one score" and "two scores." In 2016-2025, teams that started a drive down 9 to 11 points with three to four minutes left won about one game in 75; down 8, about one in 10. The samples are small, but the direction is the lesson of lumpy scoring: the three points the Falcons left on the field were worth far more than three points. Running the ball three times would also have forced New England to burn its timeouts. Atlanta's offensive coordinator and play-caller that season was Kyle Shanahan, who has called the second-down sack the play he regrets most ("I wish I had dialed something up differently") while rejecting the idea that simply running the ball would have won the game.⁷ (Two weeks later he was the 49ers' head coach; that offense is the subject of Case Study: The Shanahan Tree.)

⁶nflverse play-by-play, game 2016_21_NE_ATL: fourth quarter, 4:40, Atlanta first-and-10 at the New England 22 (Freeman run for -1); 3:56, Ryan sacked by Trey Flowers for -12; 3:50, a 9-yard completion wiped out by offensive holding; 3:38, punt. See also https://en.wikipedia.org/wiki/Super_Bowl_LI.

⁷Will Brinson, "Kyle Shanahan explains the play he regrets the most from the Falcons' Super Bowl loss," CBS Sports (quoting *The Rich Eisen Show*): <https://www.cbssports.com/nfl/news/kyle-shanahan-explains-the-play-he-regrets-the-most-from-the-falcons-super-bowl-loss>; NESN, "Kyle Shanahan Has Just One Regret From Patriots' Super Bowl LI Comeback" (January 2020), on his rejection of the "just run it" narrative: <https://nesn.com/?p=1039415>.

Leverage: which moments matter

Some plays can move win probability a lot, and most can't. **Leverage** puts a number on that. The **leverage** of a game situation is how much the next play is expected to move win probability, in either direction, relative to an average play: leverage 1 is an ordinary snap, 3 means the next play typically moves WP three times as much. (The idea, and the name "leverage index," come from baseball analyst Tom Tango; football analysts adapted it.⁸) Measuring it takes one line: average the absolute WPA of real plays in each situation, and divide by the average over all plays.

```
lv = reg[reg.play_type.isin(["pass", "run"]) & reg.wpa.notna() & reg.down.notna()].copy()
lv["abs_wpa"] = lv.wpa.abs()
lv["time"] = pd.cut(lv.game_seconds_remaining, [-1, 120, 300, 600, 900, 1800, 2700, 3600],
                    labels=["Q4\nlast 2", "Q4\n2-5", "Q4\n5-10", "Q4\n10-15", "Q3", "Q2", "Q1"])
lv["margin"] = pd.cut(lv.score_differential, [-100, -17, -9, -4, -1, 0, 3, 8, 16, 100],
                     labels=["down 17+", "down 9-16", "down 4-8", "down 1-3", "tied",
                              "up 1-3", "up 4-8", "up 9-16", "up 17+"])
LI = lv.pivot_table(index="margin", columns="time", values="abs_wpa", aggfunc="mean",
                    observed=False) / lv.abs_wpa.mean()
LI = LI[LI.columns[::-1]] # Q1 on the left, the end of the game on the right
srt = np.sort(lv.abs_wpa.to_numpy())[::-1]
top10 = srt[: len(srt) // 10].sum() / srt.sum()
li_peak = LI.loc["down 1-3"].iloc[-1] # trailing by 1-3, last two minutes
print(f"Average |WPA| per play: {lv.abs_wpa.mean():.3f}; "
      f"the top 10% of plays account for {top10:.0%} of all WP movement")
```

Average |WPA| per play: 0.025; the top 10% of plays account for 39% of all WP movement

```
cmap_lv = LinearSegmentedColormap.from_list("lv", ["#f7f6f2"] + style.SEQUENTIAL_BLUE)
fig, ax = plt.subplots(figsize=(6.5, 4.0))
ax.imshow(LI.values, cmap=cmap_lv, vmin=0, vmax=3.4, aspect="auto")
for i in range(LI.shape[0]):
    for j in range(LI.shape[1]):
        v = LI.values[i, j]
        ax.text(j, i, f"{v:.1f}", ha="center", va="center", fontsize=7.5,
                color="white" if v > 1.9 else style.INK)
ax.set_xticks(range(LI.shape[1]), LI.columns, fontsize=7.5)
ax.set_yticks(range(LI.shape[0]), LI.index, fontsize=7.5)
ax.set_xlabel("Time left")
ax.set_ylabel("Offense's margin")
ax.set_title("Leverage: how much the next play can matter")
ax.grid(False)
ax.tick_params(length=0)
plt.show()
```

⁸Tom Tango's leverage index for baseball: "Crucial Situations," *The Hardball Times* / <http://www.insidethebook.com/li.shtml>; Tom M. Tango, Mitchel G. Lichtman and Andrew E. Dolphin, *The Book: Playing the Percentages in Baseball* (2007). Brian Burke adapted it for football on Advanced NFL Stats.

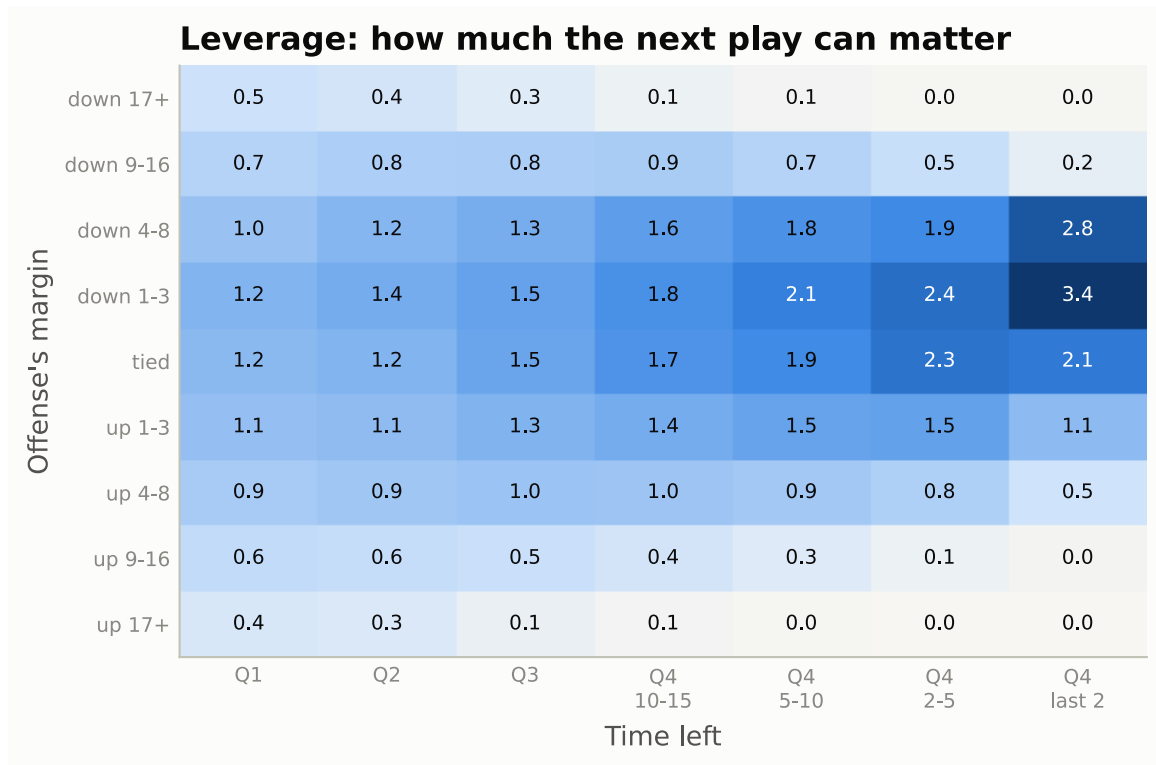


Figure 5: Leverage by score and time, 2016-2025 regular seasons: the average size of the win-probability swing on a run or pass in each situation, divided by the average over all plays (so 1.0 is an ordinary snap). Score is from the offense's point of view. Leverage climbs through the fourth quarter in one-score games and peaks when the offense trails by 1-3 in the last two minutes; it collapses toward zero in blowouts at any time. The offense leading late is lower leverage than the offense trailing, because a leading team with the ball can run the clock.

The table formalises what every fan feels. A tied game in the first quarter is about an ordinary snap. Down 1-3 with under two minutes left, a play matters about 3 times as much. Up or down 17, even the first quarter is quieter than average, and late it is essentially zero. And leverage is concentrated: the top tenth of plays carry 39% of all the win-probability movement in a season.

! Why it exists

Leverage answers a question every other number dodges: *does this play matter?* It is why analysts use the garbage-time filter (low-leverage plays are played differently and tell you little about a team), why “clutch” statistics are so noisy (the high-leverage sample is small), and why WPA, which weights every play by its leverage, is a better description of who *won* a game while EPA is a better measure of who *played* well. A quarterback's interception in a tied game with a minute left and the same interception up 24 cost about the same EPA and wildly different WPA. Which matters depends on the question: for “how good is he?” use EPA; for “what decided this game?” use WPA.

Fourth down as a decision tree

Now the decisions. On fourth down a coach has three options, and each one is a small probability tree:

- **Go for it.** With probability p you convert and keep the ball; with probability $1 - p$ the opponent takes over where you stand.
- **Kick a field goal.** With probability q you score 3 and kick off; otherwise the opponent takes over at the spot of the kick (or at its own 20, whichever is better for it).
- **Punt.** The opponent takes over somewhere downfield, on average.

Put the win probability of each outcome at the end of each branch, multiply through, and pick the largest. That is all a **fourth-down model** is: a conversion model, a field-goal model, a punt model, and a win-probability model to price the outcomes. The Clock, the Two-Minute Drill, and Fourth Down did this in expected points. Doing it in win probability lets the score and the clock in.

The ingredients

Every ingredient comes from the same ten seasons of data. Start with the two probabilities that drive everything: converting, and making the kick.

```
def fit_logit(X, y, iters=30):
    """Logistic regression by Newton's method: returns the weights."""
    w = np.zeros(X.shape[1])
    for _ in range(iters):
        p = 1 / (1 + np.exp(-X @ w))
        H = (X * (p * (1 - p))[:, None]).T @ X
        w += np.linalg.solve(H, X.T @ (y - p))
    return w

plays = reg[reg.play_type.isin(["pass", "run"]) & (reg.qb_kneel != 1) & (reg.qb_spike != 1)]
att4 = plays[plays.down == 4].assign(conv=lambda t: (t.yards_gained >= t.ydstogo).astype(float))
att3 = plays[plays.down == 3].assign(conv=lambda t: (t.yards_gained >= t.ydstogo).astype(float))

# Conversion: logistic in log(yards to go) and yards to go, fitted on fourth-down attempts.
ytg = att4.ydstogo.clip(1, 20).to_numpy()
W_CONV = fit_logit(np.column_stack([np.ones(len(ytg)), np.log(ytg), ytg]), att4.conv.to_numpy())

def p_conv(togo):
    k = np.clip(np.asarray(togo, float), 1, 20)
    return 1 / (1 + np.exp(-(W_CONV[0] + W_CONV[1] * np.log(k) + W_CONV[2] * k)))

# Field goals: logistic in kick distance, 2021-2025 (today's kickers are better than 2016's).
fgs = reg[(reg.play_type == "field_goal") & (reg.season >= 2021)]
W_FG = fit_logit(np.column_stack([np.ones(len(fgs)), fgs.kick_distance]),
                  (fgs.field_goal_result == "made").astype(float).to_numpy())

def p_fg(distance):
    return 1 / (1 + np.exp(-(W_FG[0] + W_FG[1] * np.asarray(distance, float))))

print(f"{len(att4):,} fourth-down attempts; conversion model at 1, 2, 4, 7, 10 to go: "
      + ", ".join(pct(p_conv(k)) for k in (1, 2, 4, 7, 10)))
print(f"{len(fgs):,} field-goal tries 2021-2025; make rate at 30, 40, 50, 55, 60 yards: "
      + ", ".join(pct(p_fg(k)) for k in (30, 40, 50, 55, 60)))
```

6,664 fourth-down attempts; conversion model at 1, 2, 4, 7, 10 to go: 67%, 59%, 49%, 38%, 29%
5,285 field-goal tries 2021-2025; make rate at 30, 40, 50, 55, 60 yards: 96%, 89%, 74%, 64%, 51%

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(6.6, 3.2))
k4 = att4.groupby(att4.ydstogo.clip(upper=10)).conv.agg(["mean", "size"])
k3 = att3.groupby(att3.ydstogo.clip(upper=10)).conv.mean()
xs = np.linspace(1, 10, 100)
a1.plot(xs, p_conv(xs), color=style.SERIES[0], lw=2, label="Fitted (4th down)")
a1.scatter(k4.index, k4["mean"], s=8 + k4["size"] / 25, color=style.SERIES[0], alpha=0.8,
           edgecolor=style.SURFACE, linewidth=0.6, zorder=3, label="4th-down attempts")
a1.scatter(k3.index, k3, s=14, marker="D", color=style.SERIES[1], zorder=3, label="3rd downs")
```

```

a1.set_xticks(range(1, 11), [str(k) for k in range(1, 10)] + ["10+"])
a1.set_ylim(0, 0.8)
a1.set_xlabel("Yards to go")
a1.set_ylabel("Converted")
a1.set_title("Converting", fontsize=10)
a1.legend(loc="upper right", fontsize=6.8)
fb = fgs.assign(b=(fgs.kick_distance // 3) * 3 + 1.5).groupby("b").agg(
    make=("field_goal_result", lambda s: (s == "made").mean()), n=("field_goal_result", "size"))
fb = fb[fb.n >= 15]
d_ = np.linspace(18, 66, 100)
a2.plot(d_, p_fg(d_), color=style.SERIES[2], lw=2, label="Fitted")
a2.scatter(fb.index, fb.make, s=14, color=style.SERIES[2], edgecolor=style.SURFACE,
    linewidth=0.6, zorder=3, label="Actual, 3-yd bins")
a2.set_ylim(0, 1.02)
a2.set_xlabel("Kick distance (yards)")
a2.set_title("Field goals, 2021-2025", fontsize=10)
a2.legend(loc="lower left", fontsize=6.8)
for a in (a1, a2):
    a.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
fig.tight_layout()
plt.show()

```

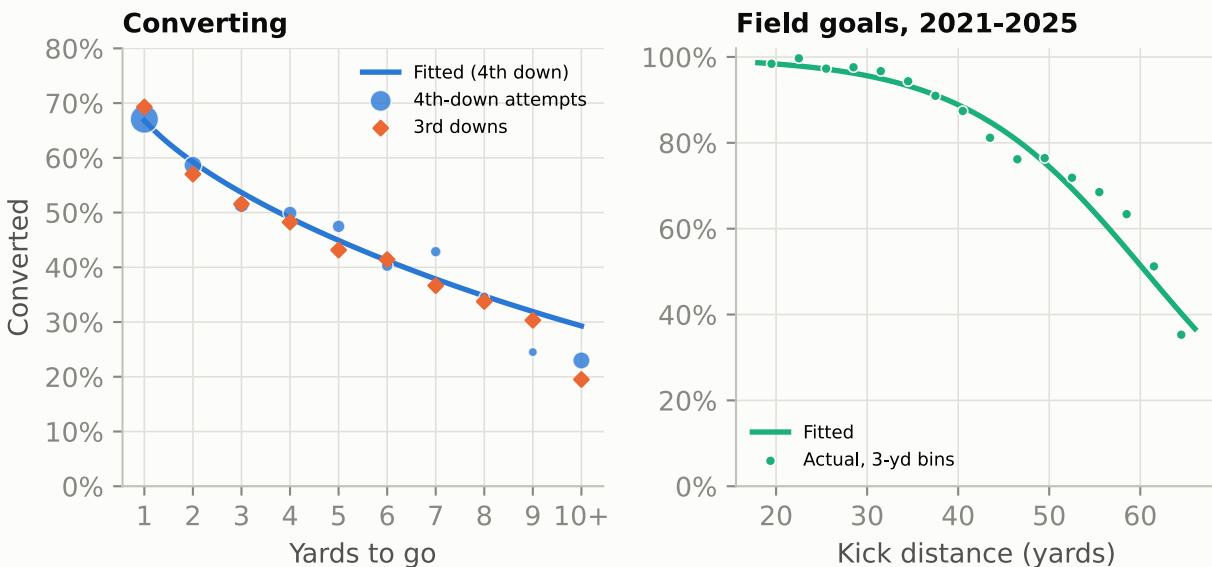


Figure 6: The two probabilities behind every fourth-down call, 2016-2025 regular seasons. Left: conversion rate by yards to go on fourth-down attempts (blue dots, sized by the number of attempts), on third downs (orange), and the fitted curve. The two downs convert at almost the same rates, which suggests teams' choice of *when* to go for it on fourth down doesn't inflate the fourth-down numbers much. Right: field-goal make rate by distance, 2021-2025, in 3-yard bins, and the fitted curve; a 46-yard try is about an 80% kick.

The third-down comparison deserves a sentence of its own. You might worry that fourth-down conversion rates are flattering, because teams go for it only when they like their chances. Third downs have no such selection (every team faces them and nearly every team runs a play), and they convert at almost exactly the same rates. Fourth-down models built by analysts often fit the conversion model on third and fourth downs together for that reason.

The remaining ingredients describe where the ball ends up. The field-goal distance is the yard line plus 18 (the 10-yard end zone and the 7 or 8 yards back to the holder). A punt's result depends on where it is kicked from; the cell measures it. Since the 2025 kickoff rules moved the touchback to the 35, a kickoff after a score gives the receiving team the ball a few yards further out than it used to; the cell measures where, from 2025 data. And a conversion usually gains more than it needed: the median conversion goes a few yards past the line to gain, and some go all the way.

```

ordered = reg.sort_values(["game_id", "play_id"])
nxt = ordered.groupby("game_id")[["yardline_100", "posteam", "down"]].shift(-1)

# Punts: where does the opponent's next snap start, by where the punt was kicked from?
pu = ordered[(ordered.play_type == "punt") & (ordered.season >= 2021)]
pn = nxt.loc[pu.index]
ok = (pn.posteam.to_numpy() != pu.posteam.to_numpy()) & pn.yardline_100.notna().to_numpy()
punt_from, punt_to = pu.yardline_100[ok].clip(35, 95), pn.yardline_100[ok]
PUNT = np.polyfit(punt_from, punt_to, 2)

def punt_spot(yl):
    """Opponent's yardline_100 after a punt from yardline_100 = yl."""
    return np.clip(np.polyval(PUNT, np.clip(yl, 35, 99)), 1, 92)

# Kickoffs after a score (2025 rules): the receiving team's first snap.
ko = ordered[(ordered.play_type == "kickoff") & (ordered.season == 2025)]
KO_SPOT = nxt.loc[ko.index].query("down == 1").yardline_100.mean()

# Yards gained beyond the line to gain on fourth-down conversions (a distribution, 0-60).
won = att4[att4.conv == 1]
EXTRA = np.bincount((won.yards_gained - won.ydstogo).clip(0, 60).astype(int), minlength=61)
EXTRA = EXTRA / EXTRA.sum()

# First-and-10 (or first-and-goal) expected points by yard line: nflverse's EP, averaged.
f10 = snaps[(snaps.down == 1) & ((snaps.ydstogo == 10) | (snaps.goal_to_go == 1))
            & (snaps.half_seconds_remaining > 120)]
EP1 = (f10.groupby("yardline_100").ep.mean().reindex(range(1, 100)).interpolate()
      .rolling(5, center=True, min_periods=1).mean())

def ep1(yl):
    return np.interp(yl, EP1.index, EP1.to_numpy())

print(f"Punt from own 30 -> opponent at its own {100 - punt_spot(70):.0f}; from midfield -> "
      f"its own {100 - punt_spot(50):.0f}; from opp 40 -> its own {100 - punt_spot(40):.0f}")
print(f"After a 2025 kickoff the receiving team starts at its own {100 - KO_SPOT:.0f}")
print(f"Median yards past the line to gain on a conversion: "
      f"{np.searchsorted(np.cumsum(EXTRA), 0.5)}")

```

```

Punt from own 30 -> opponent at its own 26; from midfield -> its own 13; from opp 40 -> its own 11
After a 2025 kickoff the receiving team starts at its own 31
Median yards past the line to gain on a conversion: 3

```

The decision function

Everything is now in place. The function below takes a fourth down (the offense's lead, its yard line, the yards to go and the minutes left) and returns the win probability of each option, plus the **break-even conversion rate**: the chance of converting at which going for it and the best kick are worth the same. It is about thirty lines, and every line is one branch of the tree.

```
YDS = np.arange(len(EXTRA))
```

```

def decide(lead, yl, togo, minutes):
    """Win probability of going for it, kicking a field goal and punting, for the offense.
    lead: offense's margin; yl: yardline_100; togo: yards to go; minutes: left in regulation.
    Arrays of fourth downs are fine: every argument broadcasts."""
    lead, yl, togo, mins = (np.atleast_1d(np.asarray(a, float)) for a in (lead, yl, togo, minutes))
    t = mins - 0.1 # the play itself takes ~6 seconds
    # G0, success: first down somewhere past the marker (or a touchdown, then kick off)
    spot = yl[:, None] - togo[:, None] - YDS[None, :]
    td = spot <= 0
    wp_td = 1 - wp_model(-(lead[:, None] + 7), ep1(K0_SPOT), t[:, None])
    wp_1st = wp_model(lead[:, None], ep1(np.clip(spot, 1, 99)), t[:, None])
    succ = (np.where(td, wp_td, wp_1st) * EXTRA[None, :]).sum(axis=1)
    # G0, failure: the opponent takes over where the ball is
    fail = 1 - wp_model(-lead, ep1(100 - yl), t)
    p = p_conv(togo)
    go = p * succ + (1 - p) * fail
    # FIELD GOAL: make -> +3 and kick off; miss -> opponent at the spot of the kick (or its 20)
    dist = yl + 18
    q = np.where(dist <= 70, p_fg(dist), 0.0)
    miss_spot = np.where(yl + 7 >= 20, 100 - (yl + 7), 80)
    fg = (q * (1 - wp_model(-(lead + 3), ep1(K0_SPOT), t))
          + (1 - q) * (1 - wp_model(-lead, ep1(miss_spot), t)))
    # PUNT: only from your own side of the opponent's 35
    punt = np.where(yl >= 35, 1 - wp_model(-lead, ep1(punt_spot(yl)), t), np.nan)
    kick = np.fmax(fg, punt) # the better kicking option
    return pd.DataFrame({"go": go, "fg": fg, "punt": punt, "kick": kick,
                        "gain": go - kick, "p_conv": p, "p_fg": q,
                        "wp_succ": succ, "wp_fail": fail,
                        "breakeven": (kick - fail) / (succ - fail)})

```

The break-even rate is the one number to carry out of this chapter. Going for it is worth $p \cdot WP_{\text{success}} + (1 - p) \cdot WP_{\text{fail}}$, and it equals the best kick when

$$p^* = \frac{WP_{\text{kick}} - WP_{\text{fail}}}{WP_{\text{success}} - WP_{\text{fail}}}.$$

The numerator is what you give up by not kicking even if you fail; the denominator is how much converting is worth over failing. If your honest estimate of *this* play against *this* defense beats p^* , go. It turns the argument from “should we go for it?”, which invites gut feeling, into “do we convert more than 46% of the time here?”, which invites evidence.

A worked example: the Lions at San Francisco, January 2024

The decision that The Clock, the Two-Minute Drill, and Fourth Down and The Modern Era both describe: the NFC Championship Game, third quarter, 7:03 left. Detroit leads San Francisco 24-10 and faces fourth-and-2 at the 49ers’ 28. A field goal would be a 46-yarder.⁹

```

cc = raw[raw.game_id == "2023_21_DET_SF"]
d1 = cc[(cc.down == 4) & (cc.posteam == "DET") & (cc.qtr == 3) & (cc.yardline_100 == 28)].iloc[0]
d2 = cc[(cc.down == 4) & (cc.posteam == "DET") & (cc.qtr == 4) & (cc.yardline_100 == 30)].iloc[0]
lions = decide([d1.score_differential, d2.score_differential], [d1.yardline_100, d2.yardline_100],
               [d1.ydstogo, d2.ydstogo],
               [d1.game_seconds_remaining / 60, d2.game_seconds_remaining / 60])

```

⁹nflverse play-by-play of the 2023 season’s NFC Championship Game (January 28, 2024, Detroit at San Francisco), game 2023_21_DET_SF: third quarter, 7:03, fourth-and-2 at the San Francisco 28 with Detroit leading 24-10, Goff incomplete; fourth quarter, 7:38, fourth-and-3 at the San Francisco 30 with San Francisco leading 27-24, Goff incomplete; final 34-31.

```
L1, L2 = lions.iloc[0], lions.iloc[1]
md_table(pd.DataFrame({
    "decision": ["4th & 2 at SF 28, up 24-10, 7:03 Q3", "4th & 3 at SF 30, down 24-27, 7:38 Q4"],
    "WP go (%)": 100 * lions.go, "WP field goal (%)": 100 * lions.fg,
    "gain from going (pts of %)": 100 * lions.gain, "break-even (%)": 100 * lions.breakeven,
    "P(convert) (%)": 100 * lions.p_conv}), widths=[22, 8, 8, 10, 8, 8])
```

decision	WP go (%)	WP field goal (%)	gain from going (pts of %)	break-even (%)	P(convert) (%)
4th & 2 at SF 28, up 24-10, 7:03 Q3	94.6	93.7	0.9	46.3	59.3
4th & 3 at SF 30, down 24-27, 7:38 Q4	43.0	36.3	6.7	34.9	53.7

```
from gridiron import Field

def tbox(ax, x, y, w, h, text, fc=style.SURFACE, ec=style.BASELINE, fs=7.6, bold=False):
    ax.add_patch(FancyBboxPatch((x - w / 2, y - h / 2), w, h, fc=fc, ec=ec, lw=0.9, zorder=3,
                                boxstyle="round,pad=0.02,rounding_size=0.12"))
    ax.text(x, y, text, ha="center", va="center", fontsize=fs, color=style.INK, zorder=4,
            fontweight="bold" if bold else "normal", linespacing=1.25)

def tarrow(ax, p0, p1, label=None, color=style.INK_2):
    ax.add_patch(FancyArrowPatch(p0, p1, arrowstyle="-|>", mutation_scale=9, color=color,
                                lw=1.0, shrinkA=0, shrinkB=0, zorder=2))

    if label:
        mx, my = (p0[0] + p1[0]) / 2, (p0[1] + p1[1]) / 2
        ax.text(mx, my + 0.16, label, ha="center", va="bottom", fontsize=7.2,
                color=style.INK, zorder=4,
                bbox=dict(boxstyle="round,pad=0.15", fc=style.SURFACE, ec="none"))

fig = plt.figure(figsize=(6.6, 6.3))
axf = fig.add_axes([0.02, 0.61, 0.96, 0.33]) # the field, Detroit moving right
axt = fig.add_axes([0.0, 0.0, 1.0, 0.57]) # the tree
LOS = 110 - 28 # BDB x of the SF 28
fld = Field(ax=axf, orient="horizontal", los=LOS, window=(-28, 38), to_go=2,
            lateral=(15, 38.3), numbers=False)
for yl in (40, 30, 20, 10): # yard labels inside the turf
    axf.text(110 - yl, 16.6, f"SF {yl}", ha="center", va="center", fontsize=6.5,
            color=style.MUTED, zorder=2)
axf.text(115, 16.6, "end zone", ha="center", va="center", fontsize=6.5, color=style.MUTED)
label_box = dict(boxstyle="round,pad=0.2", fc=style.SURFACE, ec="none", alpha=0.9)
yb = 26.65
axf.add_patch(Circle((LOS, yb), 0.9, color=style.BALL, zorder=6))
axf.add_patch(FancyArrowPatch((LOS + 1.0, yb + 1.6), (LOS + 5.0, yb + 1.6), arrowstyle="-|>",
                                mutation_scale=9, color=style.OFFENSE_DARK, lw=2, zorder=5))
axf.text(LOS + 7.0, 33.6, "convert: first down near the\nSF 23 (or a touchdown)", ha="left",
        va="center", fontsize=6.8, color=style.OFFENSE_DARK, zorder=7, bbox=label_box)
axf.text(LOS - 1.0, 19.8, "fail: 49ers' ball at their 28", ha="right", va="center",
        fontsize=6.8, color=style.DEFENSE_DARK, zorder=7, bbox=label_box)
KSPOT = LOS - 8 # holder 8 yards behind the line
axf.add_patch(Circle((KSPOT, yb), 0.6, color=style.SERIES[2], zorder=6))
```

```

axf.add_patch(FancyArrowPatch((KSPOT, yb), (118.5, yb), arrowstyle="-|>", mutation_scale=8,
                             color=style.SERIES[2], lw=1.1, ls=(0, (3, 2)),
                             connectionstyle="arc3,rad=-0.12", zorder=5))
axf.text(KSPOT - 1.0, 34.3, "field goal from the SF 36\n(46 yards); a miss gives\nSF the ball there",
         ha="right", va="center", fontsize=6.8, color=style.INK, zorder=7, bbox=label_box)
axf.set_title("4th & 2 at the SF 28 · Detroit 24, San Francisco 10 · 7:03, 3rd quarter",
             loc="left", fontsize=9, fontweight="bold", color=style.INK)

axt.set_xlim(0, 10)
axt.set_ylim(0, 6.2)
axt.set_axis_off()
GO_C, FG_C = style.OFFENSE_TINT, "#dff3ea"
tbox(axt, 1.05, 3.1, 1.9, 1.3, "4th & 2\nSF 28\nup 14, 7:03 Q3", bold=True)
tbox(axt, 4.0, 4.6, 2.2, 1.15, f"GO FOR IT\nexpected WP {pct(L1.go, 1)}", fc=GO_C, bold=True)
tbox(axt, 4.0, 1.5, 2.2, 1.15, f"FIELD GOAL\nexpected WP {pct(L1.fg, 1)}", fc=FG_C, bold=True)
tarrow(axt, (2.0, 3.45), (2.9, 4.45))
tarrow(axt, (2.0, 2.75), (2.9, 1.65))
tbox(axt, 8.1, 5.55, 3.4, 0.95, f"Convert: keep the ball\nWP {pct(L1.wp_succ, 1)}")
tbox(axt, 8.1, 3.95, 3.4, 0.95, f"Fail: SF ball at its 28,\nDET still up 14 · WP {pct(L1.wp_fail, 1)}")
make_wp = 1 - wp_model(-(d1.score_differential + 3), epl(KO_SPOT), d1.game_seconds_remaining / 60 - 0.1)
miss_wp = 1 - wp_model(-d1.score_differential, epl(100 - 35), d1.game_seconds_remaining / 60 - 0.1)
tbox(axt, 8.1, 2.15, 3.4, 0.95, f"Make: up 17, kick off\nWP {pct(float(make_wp), 1)}")
tbox(axt, 8.1, 0.55, 3.4, 0.95, f"Miss: SF ball at its 36\nWP {pct(float(miss_wp), 1)}")
tarrow(axt, (5.1, 4.85), (6.4, 5.45), pct(L1.p_conv))
tarrow(axt, (5.1, 4.35), (6.4, 4.0), pct(1 - L1.p_conv))
tarrow(axt, (5.1, 1.75), (6.4, 2.1), pct(L1.p_fg))
tarrow(axt, (5.1, 1.25), (6.4, 0.6), pct(1 - L1.p_fg))
axt.text(4.0, 3.62, f"break-even: {pct(L1.breakeven)}", ha="center", va="center", fontsize=7.4,
        color=style.OFFENSE_DARK, fontweight="bold")
axt.text(4.0, 3.05, f"go wins by {100 * L1.gain:.1f} pts of WP", ha="center", va="center",
        fontsize=7.4, color=style.INK)
plt.show()

```

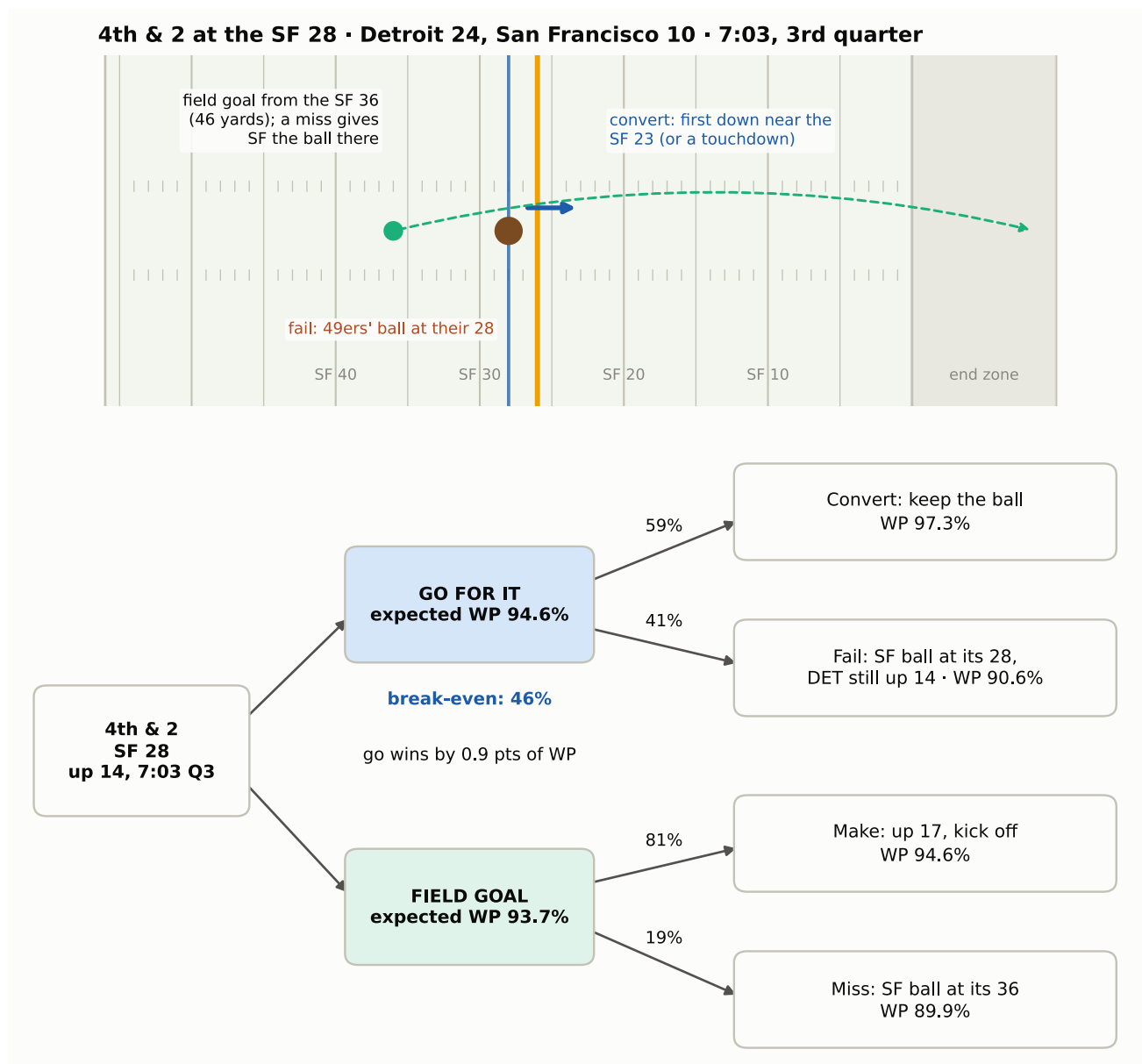


Figure 7: One fourth down as a decision tree: Detroit at San Francisco, NFC Championship Game, January 28, 2024. Top: the situation on the field, with Detroit moving right; the yellow line is the line to gain, and the markers show where each branch leaves the ball. Bottom: the tree from this chapter's model. Going for it wins on average by less than one point of win probability, because the failure branch costs little (San Francisco still needs two scores from its own 28) while conversion leaves a likely touchdown or a shorter kick. The break-even conversion rate is below the league's typical rate for fourth-and-2, so the model says go, narrowly.

Read the tree from the right. The two outcomes of going for it are far apart only in one direction: converting pushes Detroit toward a three-score lead, while failing still leaves it up 14 with San Francisco 72 yards from the end zone. The field goal's branches are *both* good but cap the upside. The break-even rate, 46%, is below the 59% that a league-average offense converts on fourth-and-2, so the model says go, by a margin of 0.9 points of win probability.

Now compare the professionals. On that fourth-and-2, ESPN's model had going for it ahead by two-tenths of a point (90.5% to 90.3%), and Next Gen Stats by one point (86.8% to 85.8%).¹⁰ Three models, built three different ways, agree on the direction and on the size: *close, lean go*. The second decision, fourth-and-3 at the 30 trailing 27-24 with 7:38 left, is a different story. ESPN called it a near toss-up (39.1% to 38.8%), Next Gen Stats leaned go by 2.5 points, and our model says go by 6.7. That disagreement is not a

¹⁰ESPN Analytics figures, as reported by NBC Sports Bay Area, "Campbell explains disastrous fourth-down decisions vs. 49ers" (January 2024): <https://www.nbcsportsbayarea.com/nfl/san-francisco-49ers/campbell-explains-disastrous-fourth-down-decisions-vs-49ers/1696885/>, and Denny Carter, NBC Sports, "Dan Campbell did nothing wrong" (January 29, 2024): <https://www.nbcsports.com/fantasy/football/news/dan-campbell-did-nothing-wrong>. Next Gen Stats (86.8 against 85.8 on the fourth-and-2; 32.7 against 30.2 on the fourth-and-3) as quoted by Ben Solak, The Ringer, "The Hot Read, Conference Championship Edition" (January 29, 2024): <https://www.theringer.com/nfl/2024/1/29/24054043/nfl-conference-championship-hot-read-detroit-lions-kansas-city-chiefs>.

rounding error. It is the signature of the part of the game where simple models are weakest: the fourth quarter, when a field goal's 3 points are worth something quite specific (a tie) and the clock starts to matter. The next sections look at both problems, where models break and how uncertain they are, after mapping the whole field.

i Madden vs. real life

In Madden you can go for it on every fourth down, and the worst that happens is a lost game. A real head coach answers for every failed attempt on Monday morning, and coaches have long been judged more harshly for a failed gamble than for a “safe” kick that loses the game a little more slowly. David Romer’s 2006 paper on fourth downs, which found teams far too conservative, suggested exactly this kind of reason: coaches may protect their jobs rather than maximise wins.¹¹ That is part of why the models led and the coaches followed only slowly; the measurement section below shows how slowly.

The recommendation map

Run `decide()` over every yard line and every distance, and the whole field becomes a map. The two panels below hold everything constant except the score and the clock: a tied game in the first quarter, and the same field with the offense leading by 4 with 8 minutes left.

```
YL = np.arange(95, 0, -5)                                # own 5 ... opponent's 5
YT = np.arange(1, 11)
cmap_go = LinearSegmentedColormap.from_list("go", [style.DIVERGING[2], style.DIVERGING[1],
                                                    style.DIVERGING[0]])

fig, axes = plt.subplots(2, 1, figsize=(6.6, 6.6))
for ax, (lead, mins, title) in zip(axes, [(0, 55, "Tied, first quarter"),
                                           (4, 8, "Leading by 4, 8:00 left in the 4th quarter")]):
    Y, T = np.meshgrid(YL, YT)                            # rows: yards to go; columns: field position
    r = decide(np.full(Y.size, lead), Y.ravel(), T.ravel(), np.full(Y.size, mins))
    gain = 100 * r.gain.to_numpy().reshape(Y.shape)
    be = r.breakeven.to_numpy().reshape(Y.shape)
    fg_better = (r.fg > r.punt.fillna(-1)).to_numpy().reshape(Y.shape)
    ax.imshow(gain, cmap=cmap_go, vmin=-6, vmax=6, aspect="auto")
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            txt = f"{100 * be[i, j]:.0f}" if be[i, j] < 1 else "-"
            ax.text(j, i, txt, ha="center", va="center", fontsize=5.6,
                    color="white" if abs(gain[i, j]) > 4 else style.INK)
    # punt -> field goal boundary, row by row
    edge = [np.argmax(fg_better[i]) if fg_better[i].any() else len(YL) for i in range(len(YT))]
    xs_, ys_ = [], []
    for i, e in enumerate(edge):
        xs_ += [e - 0.5, e - 0.5]
        ys_ += [i - 0.5, i + 0.5]
    ax.plot(xs_, ys_, color=style.INK, lw=1.3)
    ax.set_xticks(range(len(YL)), [f"own\n{100 - y}" if y > 50 else ("50" if y == 50 else f"opp\n{y}")
                                for y in YL], fontsize=6)
    ax.set_yticks(range(len(YT)), [str(k) for k in YT], fontsize=7)
    ax.set_ylabel("Yards to go", fontsize=8)
    ax.set_title(title, loc="left", fontsize=9.5, fontweight="bold")
    ax.grid(False)
    ax.tick_params(length=0)
fig.tight_layout(h_pad=1.2)
plt.show()
```

¹¹David Romer, “Do Firms Maximize? Evidence from Professional Football,” *Journal of Political Economy* 114, no. 2 (2006): 340-365, <https://doi.org/10.1086/501171>.

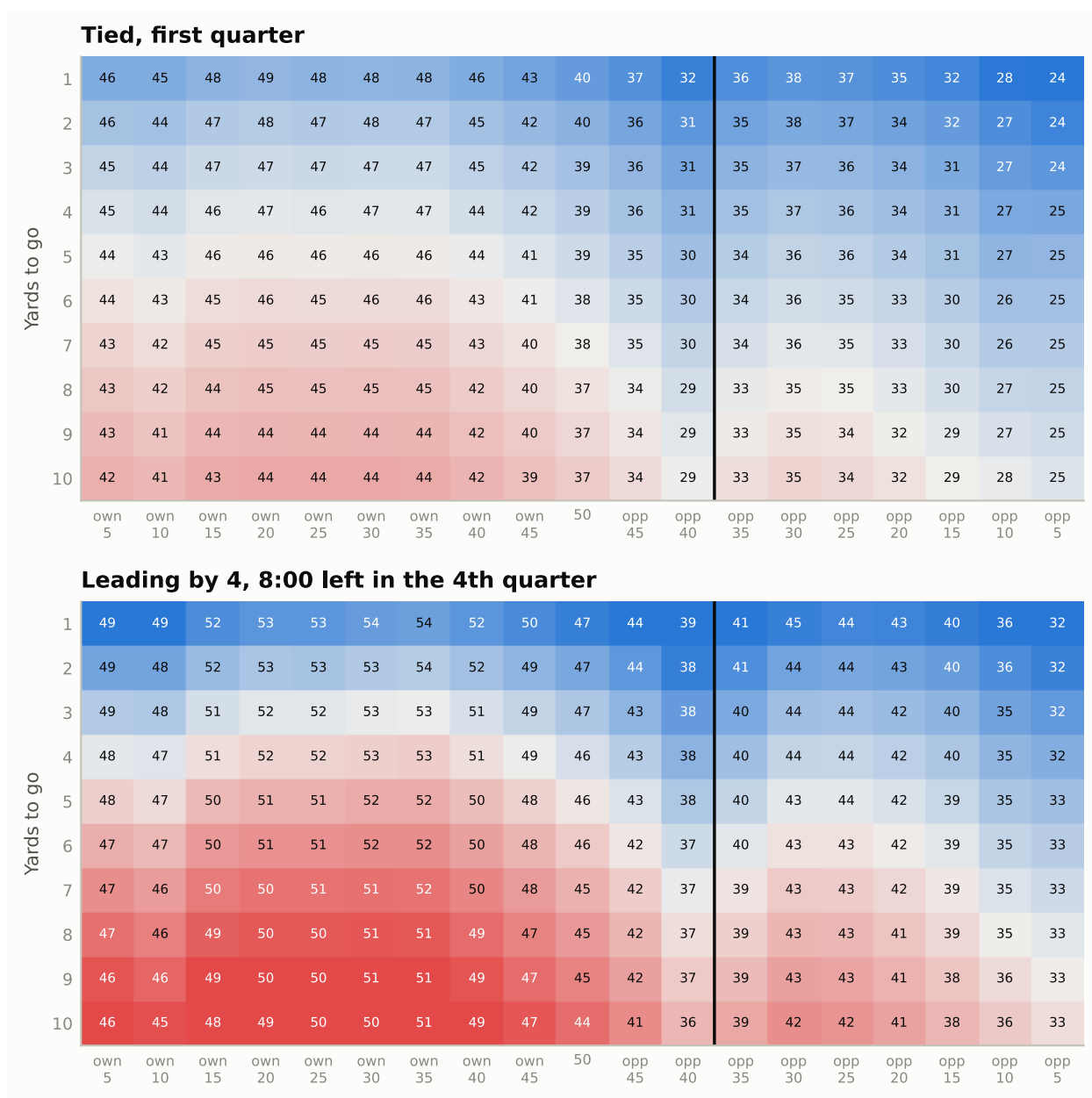


Figure 8: Fourth-down recommendations from this chapter's model. Each cell is a fourth down: yards to go down the side, field position across (the offense moving left to right). Colour is the gain in win probability from going for it over the better kick, in points of percentage (blue: go; red: kick; pale: within about a point, a toss-up). The number in each cell is the break-even conversion rate. The black step line marks where the better kick changes from a punt (left of it) to a field goal (right). Top: tied, first quarter.

Bottom: leading by 4 with 8:00 left, when field goals and punts gain value and the go zone shrinks to short yardage.

```
tie_q1 = decide([0, 0, 0, 0], [66, 50, 40, 25], [1, 2, 4, 6], [55, 55, 55, 55])
print("Tied, Q1: break-even / gain from going")
for (yl, k), (_, row) in zip([(66, 1), (50, 2), (40, 4), (25, 6)], tie_q1.iterrows()):
    print(f"  4th & {k} at yardline_100 {yl}: break-even {row.breakeven:.0%}, "
          f"P(convert) {row.p_conv:.0%}, gain {100 * row.gain:+.1f} pts")
```

```
Tied, Q1: break-even / gain from going
  4th & 1 at yardline_100 66: break-even 48%, P(convert) 67%, gain +2.9 pts
  4th & 2 at yardline_100 50: break-even 40%, P(convert) 59%, gain +3.1 pts
  4th & 4 at yardline_100 40: break-even 31%, P(convert) 49%, gain +3.0 pts
  4th & 6 at yardline_100 25: break-even 35%, P(convert) 41%, gain +1.0 pts
```

Four lessons are visible on the top map, and they match the rules of thumb in The Clock, the Two-Minute Drill, and Fourth Down:

- **Short yardage is blue almost everywhere.** On fourth-and-1 the break-even rate is in the 40s across most of the field and a league-average offense converts about two times in three, so going is right even in your own territory. On fourth-and-1 at your own 34 the model's gain is 2.9 points, enough that punting there is one of the most expensive routine choices in football.
- **The break-even rate falls as you approach the end zone.** In your own half, failure hands the opponent a short field, so you need to convert about half the time. Near the goal line failure leaves the opponent pinned, and the break-even rate drops into the 20s.
- **The opponent's 30 to 40 is the go zone.** Too long for a sure field goal and too short for a useful punt, it is the strip where even fourth-and-4 or 5 says go.
- **Long yardage in your own half leans red (kick).** Fourth-and-8 at your own 30 needs a conversion rate in the mid-40s, and nobody converts fourth-and-8 that often.

The bottom map shows what win probability adds to the expected-points version. Leading by 4 late, a field goal turns a one-score lead into a 7-point one and a punt makes the opponent go the length of the field against the clock; both kicks gain value, and the blue region shrinks to short yardage. The mirror image is also true (try `decide(-4, ...)` yourself): trailing late, kicks lose value, because a field goal doesn't get you level, and the blue region spreads.

Where the simple model breaks, and how sure any model is

The last five minutes

Use the model on the most famous fourth down of the analytics era: Bill Belichick's fourth-and-2 at his own 28, leading the Colts 34-28 with 2:08 left in November 2009, which *Dynasties* and *Counterpunches* tells in full.

```
b09 = decide([6], [72], [2], [2 + 8 / 60]).iloc[0]
print(f"Random-walk model: go {b09.go:.1%}, punt {b09.punt:.1%} -> says PUNT by "
      f"{100 * (b09.punt - b09.go):.1f} points")

# What actually happens: drives that start in the last 1:30-2:30, trailing by 4-8 (a TD needed)
first_snaps = (reg[reg.down.notna()].sort_values(["game_id", "play_id"])
               .groupby(["game_id", "drive"]).first())
late = first_snaps[(first_snaps.qtr == 4) & first_snaps.game_seconds_remaining.between(90, 150)
                  & first_snaps.score_differential.between(-8, -4)
                  & first_snaps.yardline_100.between(60, 80)]
td_after_punt = (late.fixed_drive_result == "Touchdown").mean()
print(f"Drives needing a TD from their own 20-40 with 1:30-2:30 left scored one "
      f"{td_after_punt:.0%} of the time ({len(late)} drives, 2016-2025)")
```

```
Random-walk model: go 84.5%, punt 86.3% -> says PUNT by 1.8 points
Drives needing a TD from their own 20-40 with 1:30-2:30 left scored one 28% of the time (155 drives, 2016-2025)
```

Our model says punt, by about 2 points. Burke's 2009 analysis said go, by about nine, and the drive data show why ours is wrong here. After a punt the Colts would need a touchdown, not "some points," from around their own 30 with two minutes left. Teams in that spot in 2016-2025 scored one about 28% of the time, close to the 30% Burke used. Our random walk instead sees an effective deficit of about four and a half points with two minutes to make it up, and it can't tell that a field goal is worthless to Indianapolis. On the failure branch it makes the opposite mistake: Burke estimated that teams score a touchdown from about 30 yards out with two minutes left a little over half the time, while our model, seeing a 6-point deficit and a possession worth about four points, gives the Colts only about a third. Both errors favour the punt.

So the simple model is honest about the first 55 minutes (Figure 3) and unreliable in the last five, where the lumps in the scoring and the clock decide games. That is the job of the inputs we left out: timeouts, the exact time, and a model flexible enough to learn that "down 6 with 2 minutes" is nothing like "down 4.5 with 2 minutes." It is also why everything the next section measures uses the model only **outside the last five minutes of the game and the last two of the first half**.

How uncertain is the advice?

A fourth-down recommendation is an estimate built on estimates, so it has error bars. Three kinds of uncertainty are worth separating, and they are very different in size.

1. Sampling noise in the league-average inputs: small. Refit the conversion and field-goal models on resampled data (a bootstrap: draw the same number of plays at random, with replacement, and refit) and recompute the decision each time.

```
rng = np.random.default_rng(1303)
X4 = np.column_stack([np.ones(len(ytg)), np.log(ytg), ytg]); y4 = att4.conv.to_numpy()
XF = np.column_stack([np.ones(len(fgs)), fgs.kick_distance]); yF = (fgs.field_goal_result ==
"made").to_numpy(float)
keep = (W_CONV.copy(), W_FG.copy())
gains = []
for _ in range(200):
    i, j = rng.integers(0, len(y4), len(y4)), rng.integers(0, len(yF), len(yF))
    W_CONV, W_FG = fit_logit(X4[i], y4[i]), fit_logit(XF[j], yF[j])
    gains.append(100 * decide(d1.score_differential, d1.yardline_100, d1.ydstogo,
                             d1.game_seconds_remaining / 60).gain[0])
W_CONV, W_FG = keep # restore the fitted models
lo, hi = np.percentile(gains, [5, 95])
print(f"Detroit's 4th & 2: gain from going {100 * L1.gain:.2f} pts; "
      f"bootstrap 90% interval {lo:.2f} to {hi:.2f}")
```

Detroit's 4th & 2: gain from going 0.87 pts; bootstrap 90% interval 0.77 to 0.96

With thousands of attempts and kicks behind them, league-average conversion and make rates are known to within a point or two, and the recommendation barely moves.

2. This team, this play, this defense: large. The model assumes a league-average offense against a league-average defense. Detroit's 2023 offense was one of the league's best, and San Francisco's defense was one of its best too. Nobody knows the true conversion probability of *that* play to better than several points either way, and the break-even rate tells you how much that matters: the call flips if you think Detroit converts less than 46% of the time instead of the league's 59%. That is a judgement about football, not statistics, and it is the coach's real job on fourth down.

3. The model itself: larger than the media admit. Different reasonable models (ours, ESPN's, Next Gen Stats', nflverse's) disagree by a point or more, as the Detroit example showed, and the disagreement grows late in games. Ryan Brill, Ron Yurko and Abraham Wyner made this point formally in a 2023 paper with a pointed title, "Analytics, have some humility": when they bootstrapped the win-probability model itself, resampling whole games rather than plays, the uncertainty in the recommended decision was far larger than the confident "+3% go" graphics on television imply, and many close calls were genuinely uncertain.¹²

Common misconception

"The bot says go, so going for it is right." The bot says that *for an average team in this situation*, going for it has the higher expected win probability, by some margin, under one model. When the margin is half a point, a better defense, a worse kicker or a different model can flip it. When the margin is five points, it almost never flips. Read the size of the gain, not just its sign, and save the outrage for big gains that coaches ignore.

The fourth-down bots

Analysts have argued about fourth down for half a century. In 1978 Virgil Carter, the quarterback who built the first expected-points table (see 13-02), and Robert Machol published "Optimal Strategies on Fourth Down," arguing from their table that teams kicked too often near the opponent's goal line.¹³ David Romer's 2006 paper made the case for the whole field with modern data,

¹²Ryan S. Brill, Ronald Yurko and Abraham J. Wyner, "Analytics, have some humility: a statistical view of fourth-down decision making," arXiv:2311.03490 (first posted November 2023; revised January 2025): <https://arxiv.org/abs/2311.03490>.

¹³Virgil Carter and Robert E. Machol, "Optimal Strategies on Fourth Down," *Management Science* 24, no. 16 (1978): 1758-1762, <https://doi.org/10.1287/mnsc.24.16.1758>.

and concluded that teams' fourth-down choices cost them wins every season (see the Madden callout above). Coaches mostly ignored both.

What changed the conversation was making the model *public and live*. In 2013 The New York Times launched the **4th Down Bot**, built with Brian Burke on his expected-points and win-probability models. It graded every fourth down of every game in real time on the Times' site and on Twitter, maximising expected points for most of the game and switching to win probability with about ten minutes left in the fourth quarter; when Burke joined ESPN in 2015, the Times rebuilt it.¹⁴ The bot did something no paper could: it made the argument every Sunday, about the decision fans had just watched.

The modern public standard is Ben Baldwin's model, introduced in a 2020 article in The Athletic and published as the open-source R package **nfl4th**, with a web calculator and a Twitter bot that posts its verdict on fourth downs during games. Its pieces are the ones you just built (a go-for-it model that predicts the yards gained, a punt model, a field-goal model, all priced by nflfastR's win-probability model), and its documentation is candid about what it leaves out: the identity of the kicker, punter and returner, the weather, blocked kicks and turnovers returned for touchdowns.¹⁵ Teams build their own versions too: by the early 2020s most front offices employed analysts whose work includes fourth-down and two-point recommendations for the head coach.

! Why it exists

A coach has about forty seconds to make a fourth-down decision, under pressure, with his job attached. The math is not hard, but it is easy to get wrong at that speed, and human intuition is biased toward the option that feels safe and is blamed less. A fourth-down model does the arithmetic in advance, for every situation, so the forty seconds can be spent on the one thing the model doesn't know: how good this play is against this defense. The trade-off is the one above: the model is only as good as its inputs, and it knows nothing about your team.

Measuring the coaches

With a decision function you can grade every fourth down in a decade. The cell runs `decide()` on every 2016-2025 regular-season fourth down outside the last five minutes of the game and the last two of the first half, and compares what the model recommended with what the team did. A fourth down is a **clear go** if going beats the better kick by at least one point of win probability, a **clear kick** if the kick wins by at least one point, and a **toss-up** otherwise. (One point is a choice, not a law; it keeps us out of the zone where model error swamps the answer.)

```
f4 = reg[(reg.down == 4) & reg.play_type.isin(["pass", "run", "punt", "field_goal"])
        & (reg.qb_kneel != 1) & (reg.qb_spike != 1)
        & (reg.half_seconds_remaining > 120) & (reg.game_seconds_remaining > 300)].copy()
res = decide(f4.score_differential, f4.yardline_100, f4.ydstogo, f4.minutes)
res.index = f4.index
f4 = f4.join(res)
f4["went"] = f4.play_type.isin(["pass", "run"]).astype(float) # fakes count as going for it
f4["call"] = np.select([f4.gain >= 0.01, f4.gain <= -0.01], ["clear go", "clear kick"], "toss-up")
chosen = np.select([f4.went == 1, f4.play_type == "punt"], [f4.go, f4.punt], f4.fg)
f4["wp_lost"] = (f4[["go", "fg", "punt"]].max(axis=1) - chosen).clip(lower=0)
f4["wp_lost"] = f4.wp_lost.where(f4.wp_lost >= 0.01, 0.0) # count clear mistakes only

trend = f4.pivot_table(index="season", columns="call", values="went", aggfunc="mean")
share = f4.call.value_counts(normalize=True)
lost_pg = (f4.groupby(["season", "posteam"]).wp_lost.sum()
           / f4.groupby(["season", "posteam"]).game_id.nunique().groupby("season").mean())
```

¹⁴Nieman Lab, "Yes, the Vikings should have gone for it on 4th down, and a New York Times robot knows why" (December 2013): <https://www.niemanlab.org/2013/12/yes-the-vikings-should-have-gone-for-it-on-4th-down-and-a-new-york-times-robot-knows-why/>; Nieman Lab, "Faster, stronger, and with better kickers: The New York Times 4th Down Bot gets an upgrade" (October 2015), on the rebuild after Burke moved to ESPN and on the switch from expected points to win probability late in games: <https://www.niemanlab.org/2015/10/faster-stronger-and-with-better-kickers-the-new-york-times-4th-down-bot-gets-an-upgrade/>.

¹⁵Ben Baldwin, "NFL fourth-down decisions: The math behind the league's new aggressiveness," The Athletic, October 28, 2020; the **nfl4th** R package (go-for-it, punt and field-goal models; listed limitations), <https://github.com/nflverse/nfl4th> and its README on CRAN, <https://cloud.r-project.org/web/packages/nfl4th/readme/README.html>; the bot's account (@ben_bot_baldwin) is described in Brill, Yurko and Wyner (2023).

```
print(f"{len(f4):,} fourth downs graded: " + ", ".join(f"{k} {v:.0%}" for k, v in share.items()))
print(f"Went for it on clear-go fourth downs: {trend.loc[2016, 'clear go']:.0%} in 2016, "
      f"{trend.loc[2025, 'clear go']:.0%} in 2025")
print(f"WP given up per team-game on clear mistakes: {100 * lost_pg.loc[2016]:.1f} points "
      f"(2016), {100 * lost_pg.loc[2025]:.1f} (2025)")
```

29,918 fourth downs graded: clear go 35%, toss-up 33%, clear kick 32%
 Went for it on clear-go fourth downs: 19% in 2016, 41% in 2025
 WP given up per team-game on clear mistakes: 5.0 points (2016), 3.0 (2025)

```
fig, ax = plt.subplots(figsize=(6.5, 3.5))
for k, (c, name) in enumerate([("clear go", "Model: clear go"), ("toss-up", "Model: toss-up"),
                              ("clear kick", "Model: clear kick")]):
    ax.plot(trend.index, trend[c], color=style.SERIES[k], lw=2, marker="o", ms=4, label=name)
    ax.text(2025.25, trend.loc[2025, c], pct(trend.loc[2025, c]), fontsize=7.5,
           color=style.SERIES[k], va="center", fontweight="bold")
ax.set_xlim(2015.6, 2025.9)
ax.set_ylim(0, 0.55)
ax.set_xticks(range(2016, 2026))
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
ax.set_ylabel("Went for it")
ax.set_title("Coaches moved toward the model, about halfway")
ax.legend(loc="upper left", fontsize=7.5)
plt.show()
```

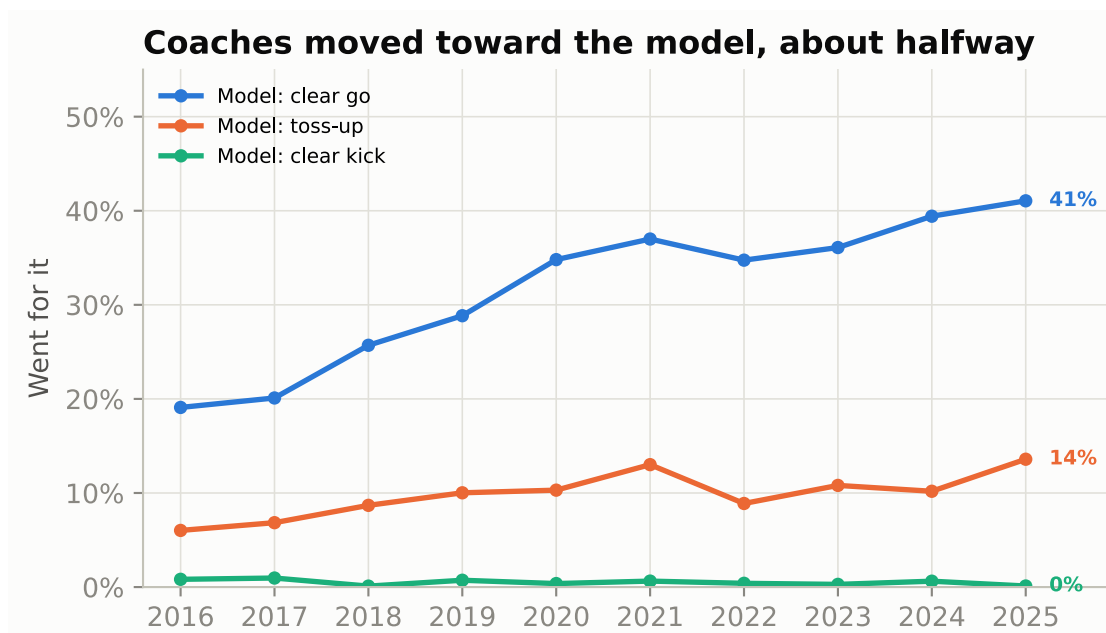


Figure 9: How often teams went for it on fourth down, split by what this chapter’s model recommended, 2016-2025 regular seasons (excluding the last five minutes of games and the last two of the first half). On clear-go fourth downs (going wins by at least a point of win probability) the go rate roughly doubled, from about one in five to about two in five. On toss-ups it also roughly doubled, from a low base. On clear-kick fourth downs, teams almost never go for it, and never did: the conservatism the models found was all on one side.

The chart is the aggression shift that The Clock, the Two-Minute Drill, and Fourth Down and The Modern Era describe, seen through the model. Three things stand out. The movement is **real but incomplete**: even in 2025, teams punted or kicked on more than half of the fourth downs where the model saw a clear gain from going. It is **one-sided**: teams almost never go for it when the model says kick, so the decades of “mistakes” were all conservative ones. And it **paid**: counting only clear mistakes (a choice

that trailed the best option by at least a point), the average team gave up about 5.0 points of win probability per game in 2016 and about 3.0 in 2025. Over a 17-game season the difference is about 0.3 wins a team, and the teams that changed first collected it first. Treat the size with caution: these are the teaching model's numbers, and it is somewhat more aggressive than the published models. Fuller models differ in size, not in direction.

Team by team

```
t3 = f4[f4.season.between(2023, 2025)]
teams = pd.DataFrame({
    "follow": t3[t3.call == "clear go"].groupby("posteam").went.mean(),
    "lost_pg": t3.groupby("posteam").wp_lost.sum() / t3.groupby("posteam").game_id.nunique(),
    "n_go_spots": t3[t3.call == "clear go"].groupby("posteam").size()})
det_by_year = f4[(f4.posteam == "DET") & (f4.call == "clear go")].groupby("season").went.mean()
lg_by_year = f4[f4.call == "clear go"].groupby("season").went.mean()
det_lost_rank = int(teams.lost_pg.rank().loc["DET"])
det_rank = (f4[(f4.call == "clear go") & f4.season.between(2021, 2025)]
            .groupby(["season", "posteam"]).went.mean().groupby("season")
            .rank(ascending=False).xs("DET", level="posteam"))
print(teams.sort_values("follow", ascending=False).head(5).round(3))
print("Detroit's rank on clear-go follow rate, 2021-2025:", det_rank.astype(int).to_dict())
```

	follow	lost_pg	n_go_spots
posteam			
DET	0.670	0.016	91
BUF	0.580	0.016	81
PHI	0.518	0.018	85
CAR	0.496	0.031	117
DAL	0.475	0.027	101

Detroit's rank on clear-go follow rate, 2021-2025: {2021: 5, 2022: 4, 2023: 1, 2024: 1, 2025: 1}

```
fig, ax = plt.subplots(figsize=(6.5, 4.2))
others = teams.drop(index="DET")
ax.scatter(100 * others.follow, 100 * others.lost_pg, s=28, color=style.MUTED, alpha=0.75,
           edgecolor=style.SURFACE, linewidth=0.6, zorder=3)
ax.scatter(100 * teams.loc["DET", "follow"], 100 * teams.loc["DET", "lost_pg"], s=60,
           color=style.SERIES[0], edgecolor=style.SURFACE, linewidth=0.8, zorder=4)
LABEL = {"DET": (-0.9, 0.17, "right"), "BUF": (0.9, 0.0, "left"), "PHI": (0.9, 0.0, "left"),
         "CAR": (0.9, 0.0, "left"), "LAC": (0.9, -0.05, "left"), "PIT": (0.9, 0.0, "left"),
         "SEA": (-0.9, 0.0, "right"), "BAL": (0.9, 0.0, "left"), "KC": (0.9, 0.0, "left")}
for tm, (dx, dy, ha) in LABEL.items():
    ax.text(100 * teams.loc[tm, "follow"] + dx, 100 * teams.loc[tm, "lost_pg"] + dy, tm,
           fontsize=7.5, ha=ha, va="center",
           color=style.SERIES[0] if tm == "DET" else style.INK_2,
           fontweight="bold" if tm == "DET" else "normal")
coef = np.polyfit(100 * teams.follow, 100 * teams.lost_pg, 1)
xx = np.array([25, 69])
ax.plot(xx, np.polyval(coef, xx), color=style.BASELINE, lw=1, ls=(0, (4, 3)), zorder=1)
ax.set_xlim(22, 72)
ax.set_ylim(1.0, 5.6)
ax.set_xlabel("Went for it on clear-go fourth downs (%)")
ax.set_ylabel("WP given up per game (pts)")
ax.set_title("Follow the model, keep the win probability")
plt.show()
```

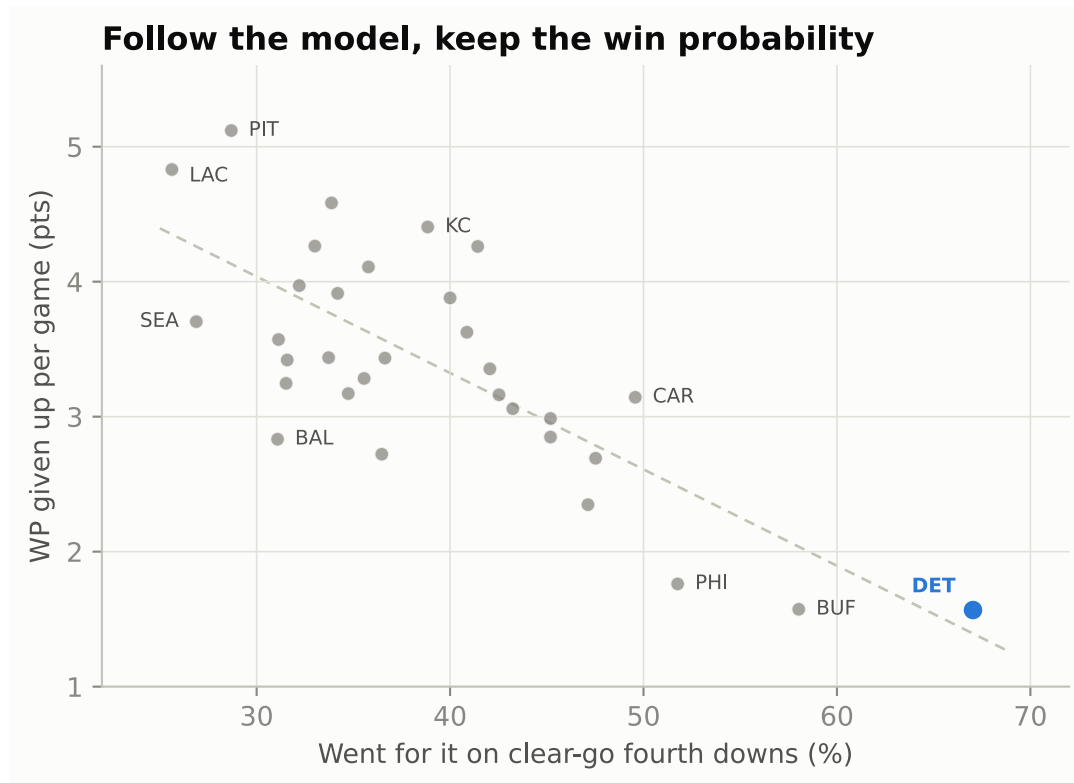


Figure 10: Team fourth-down decisions against this chapter’s model, 2023-2025 regular seasons combined. Across: the share of clear-go fourth downs on which the team went for it. Up: win probability given up per game by clear mistakes on fourth down, choices that trailed the best option by at least a point (points of percentage; lower is better). The two measures line up: the teams that followed the model most gave away least. Detroit (highlighted) followed it most often. Each team had roughly 70-110 clear-go fourth downs in the three seasons, so differences of a few points are within the noise.

i Film room: Lions 2021-2025, Dan Campbell against the model

Dan Campbell’s reputation for fourth-down aggression is documented in Case Study: The Lions by go rates and points. The model lets you ask a sharper question: when the numbers said go, did he? On clear-go fourth downs Detroit went for it 50% of the time in 2021 (the league: 37%), and 67% in 2024 (league: 39%). Its rank on that measure, season by season from 2021 to 2025: 5, 4, 1, 1, 1. Over 2023-2025 combined Detroit followed the model on 67% of clear-go fourth downs, the highest rate in the league, and ranked 1 of 32 (1 = fewest) in win probability given up per game to clear fourth-down mistakes in this chapter’s grading.

What to notice: following the model is not the same as going for it all the time. Detroit’s go rate on clear-*kick* fourth downs was still near zero. Campbell’s aggression was aimed at the short- yardage, middle-of-the-field fourth downs where the math is lopsided, and the two decisions from the NFC Championship Game that defined his reputation for critics were, by every model, among the *closest* calls he made. The fair grade is the whole season’s ledger, not the two snaps everyone remembers.

Two-point decisions: down 14, down 8

The two-point decision is the cleanest decision tree in football, because it is one play with two outcomes and a short future. Since the extra point moved back in 2015, the try has been close to a coin flip on expected points:

```
tries = reg[(reg.two_point_attempt == 1) | (reg.extra_point_attempt == 1)].copy()
tries["two"] = (tries.two_point_attempt == 1).astype(float)
P2 = (tries.loc[tries.two == 1, "two_point_conv_result"] == "success").mean()
PX = (tries.loc[tries.two == 0, "extra_point_result"] == "good").mean()
print(f"2016-2025: two-point tries succeeded {P2:.1%} of the time ")
```

```
f"({int(tries.two.sum()):,} tries) = {2 * P2:.2f} points; "
f"extra points good {PX:.1%} = {PX:.2f} points")
```

2016-2025: two-point tries succeeded 47.8% of the time (1,225 tries) = 0.96 points; extra points good 94.4% = 0.94 points

When the averages are equal, the score decides, and the most debated score is this one: **you were down 14, you just scored a touchdown, and you are now down 8 before the try**. The traditional move is to kick (down 7) and plan to kick again after the next touchdown to force overtime. The analytics move is to go for two now. Here is the whole tree, assuming you score the second touchdown and the opponent doesn't score again (the only world in which this decision matters), and that overtime is a coin flip.

- **Kick first.** Make it (probability k) and you're down 7; after the next touchdown you are down 1, and you take the better of kicking for overtime ($k \times \frac{1}{2}$) or going for the win (p). Miss it, and you're down 8; after the next touchdown you are down 2 and must go for two to reach overtime ($p \times \frac{1}{2}$).
- **Go for two first.** Make it (probability p) and you're down 6; the next touchdown ties the game and the extra point wins it (k , or overtime if it misses). Miss it, and you're down 8; after the next touchdown you are down 2 and go for two to tie ($p \times \frac{1}{2}$).

$$W_{\text{kick first}} = k \cdot \max\left(\frac{k}{2}, p\right) + (1 - k) \frac{p}{2}, \quad W_{\text{go first}} = p \left(k + \frac{1 - k}{2}\right) + (1 - p) \frac{p}{2}$$

```
def w_kick_first(p, k=PX, ot=0.5):
    return k * np.maximum(k * ot, p) + (1 - k) * p * ot

def w_go_first(p, k=PX, ot=0.5):
    return p * (k + (1 - k) * ot) + (1 - p) * p * ot

ps = np.linspace(0.25, 0.65, 401)
breakeven_p = ps[np.argmax(w_go_first(ps) > w_kick_first(ps))]
print(f"At p = {P2:.3f}: kick first {w_kick_first(P2):.1%}, go first {w_go_first(P2):.1%}; "
      f"going first is better whenever p > {breakeven_p:.3f}")
```

At p = 0.478: kick first 46.4%, go first 58.9%; going first is better whenever p > 0.352

```
fig, ax = plt.subplots(figsize=(6.5, 3.5))
ax.plot(ps, w_go_first(ps), color=style.SERIES[0], lw=2, label="Go for two after the first TD")
ax.plot(ps, w_kick_first(ps), color=style.SERIES[1], lw=2, label="Kick first (then the better choice)")
ax.axvline(P2, color=style.BASELINE, lw=1, ls=(0, (3, 2)))
ax.text(P2 + 0.004, 0.27, f"league 2016-25:\n{pct(P2, 1)}", fontsize=7.5, color=style.INK_2, va="bottom")
ax.axvline(breakeven_p, color=style.BASELINE, lw=1)
ax.text(breakeven_p - 0.004, 0.62, f"break-even\n{pct(breakeven_p)}", fontsize=7.5,
        color=style.INK_2, ha="right", va="top")
for f, c, dx, dy, ha, va in [(w_go_first, style.SERIES[0], -0.012, 0.02, "right", "bottom"),
                             (w_kick_first, style.SERIES[1], 0.008, -0.022, "left", "top")]:
    ax.scatter([P2], [f(P2)], s=26, color=c, zorder=4, edgecolor=style.SURFACE)
    ax.text(P2 + dx, f(P2) + dy, pct(f(P2)), fontsize=7.5, color=c, fontweight="bold",
            ha=ha, va=va)
ax.set_xlim(0.25, 0.65)
ax.set_ylim(0.25, 0.72)
fmt = plt.FuncFormatter(lambda v, _: f"{v:.0%}")
ax.xaxis.set_major_formatter(fmt)
ax.yaxis.set_major_formatter(fmt)
```

```

ax.set_xlabel("Your two-point success rate")
ax.set_ylabel("Win probability, given two TDs")
ax.set_title("Down 14: go for two on the first touchdown")
ax.legend(loc="upper left", fontsize=7.5)
plt.show()

```

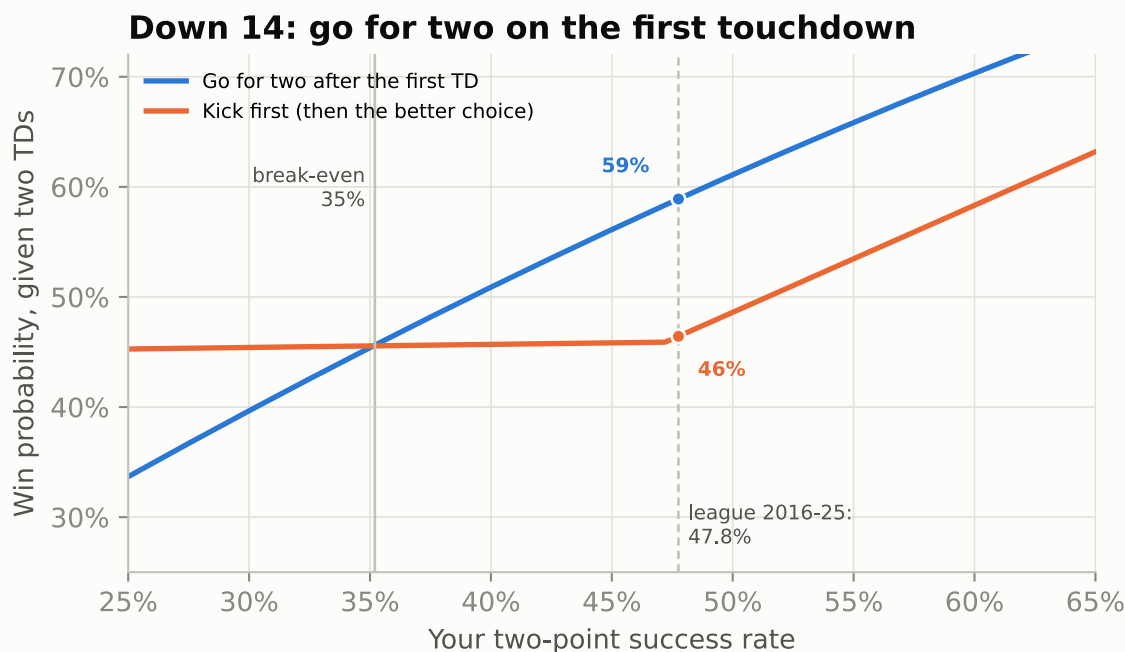


Figure 11: Down 14, you score a touchdown: the chance of winning with each plan, assuming you score the second touchdown and stop the opponent (overtime counted as a coin flip, extra points good 94% of the time). Going for two after the first touchdown beats kicking whenever your two-point success rate is above about 35%; at the league's 2016-2025 rate it is worth about 12 extra points of win probability in the games where you score twice. The reason: going early turns the second touchdown into a win (if the first try worked) or tells you in advance that you need two (if it didn't), instead of aiming the whole comeback at overtime.

The tree makes the logic visible. Kicking twice aims the entire comeback at overtime, which you win half the time. Going early gives you a roughly even chance of turning the second touchdown into an outright win, and when the first try fails you still have the overtime route, just with a two-pointer attached, and you *know* it while there is still time to plan the last drive. That knowledge is a bonus the formula doesn't count: a team that knows it needs two scores can play the rest of the game accordingly (going for it on fourth down, for instance), while the kick-first team learns about a missed extra point too late to adjust.

Two honest caveats. First, the formula assumes **two more touchdowns and nothing else**. If the opponent might score again, or you might settle for a field goal, the comparison shifts, and with more time left the margin will change before it matters, which is why the down-8 case is a fourth-quarter question. Second, the 50% overtime is a simplification: under the 2025 rule both teams get a possession in regular-season overtime, and a tie is possible.

The same reasoning settles **down 8** on its own: if you score a touchdown when trailing by 8, you are down 2 and must go for two to tie; there is no decision at all, which is exactly why the down-14 team prefers to find out early. Have coaches absorbed this? The data say yes, and quickly.

```

d8 = tries[tries.score_differential == -8]
d8_rate = pd.DataFrame({"Fourth quarter": d8[d8.qtr == 4].groupby("season").two.mean(),
                        "Any quarter": d8.groupby("season").two.mean()})
d8_n = d8[d8.qtr == 4].groupby("season").size()
fig, ax = plt.subplots(figsize=(6.5, 3.3))
for k, c in enumerate(d8_rate.columns):
    ax.plot(d8_rate.index, d8_rate[c], color=style.SERIES[k], lw=2, marker="o", ms=4, label=c)
ax.text(2025.2, d8_rate.loc[2025, "Fourth quarter"], pct(d8_rate.loc[2025, "Fourth quarter"]),

```

```

        fontsize=7.5, color=style.SERIES[0], va="center", fontweight="bold")
ax.set_xlim(2015.6, 2025.9)
ax.set_ylim(0, 0.65)
ax.set_xticks(range(2016, 2026))
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v:.0%}"))
ax.set_ylabel("Went for two at down 8")
ax.set_title("Down 8 after the touchdown: from never to half the time")
ax.legend(loc="upper left", fontsize=7.5)
plt.show()

```

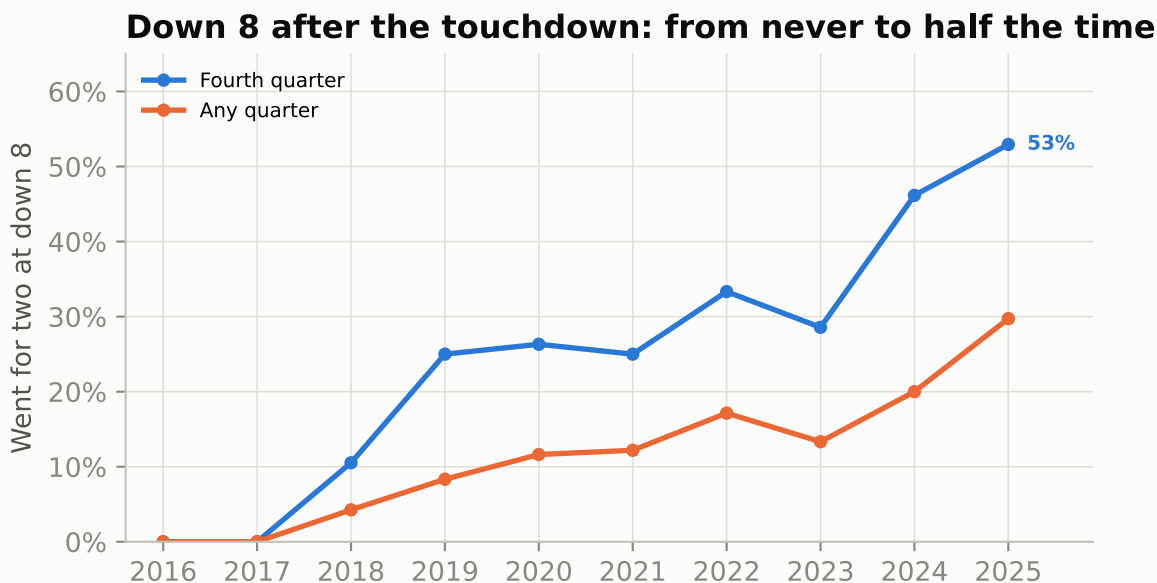


Figure 12: How often teams went for two when trailing by 8 right after a touchdown (the down-14 situation), 2016-2025 regular seasons, for tries in the fourth quarter and in any quarter. Nobody did it in 2016 or 2017; in 2025 about half of fourth-quarter tries at down 8 were two-point attempts. The samples are small (about 12 to 21 fourth-quarter tries a season), so single seasons bounce around; the trend does not.

Timeouts: pricing the clock

The third decision in this chapter's title is the quietest. The Clock, the Two-Minute Drill, and Fourth Down showed what a timeout buys: a stop of the clock that saves 30 to 40 seconds when the other team wants it running. What is that worth in wins? Our random-walk model can't say (it doesn't know timeouts exist), but nflverse's model can, because timeouts are among its inputs. Take every snap in the last 1:00-2:30 of a one-score game and ask how nflverse's win probability moves with each team's timeouts, holding the score, field position, down, distance and clock fixed with a regression.

```

lt = reg[(reg.qtr == 4) & reg.game_seconds_remaining.between(60, 150) & reg.down.notna()
        & reg.score_differential.between(-8, 8) & reg.wp.notna()]
X = np.column_stack([np.ones(len(lt)), lt.posteam_timeouts_remaining, lt.defteam_timeouts_remaining,
                    lt.score_differential, lt.yardline_100, lt.game_seconds_remaining,
                    lt.down, lt.ydstogo])
coef_to = np.linalg.lstsq(X, lt.wp.to_numpy(), rcond=None)[0]
print(f"{len(lt):,} snaps; nflverse WP per timeout: offense's own {100 * coef_to[1]:+.1f} pts, "
      f"defense's {100 * coef_to[2]:+.1f} pts")

```

8,639 snaps; nflverse WP per timeout: offense's own +1.7 pts, defense's -1.1 pts

In the last two minutes of a one-score game, each timeout in the offense's pocket is worth about 1.7 points of win probability to it in nflverse's model, and each of the defense's about 1.1 points against it (a straight-line average over many different situations; for

a team trailing by a field goal at its own 25, the value is larger). Those are modest numbers, and they explain two habits analysts criticise. **Burning a timeout early** to avoid a delay of game, or to think about a fourth down, costs about a point of win probability that will be badly missed later; it is often worth taking the 5-yard penalty instead. And **finishing a close loss with timeouts unused** is pure waste: a timeout's whole value is in being spent.

The most theatrical timeout has the weakest case. **Icing the kicker** means calling a timeout just before the opponent's late field-goal try, hoping the wait unsettles him.

```
fgl = reg.sort_values(["game_id", "play_id"])
prev_def_to = (fgl.timeout.shift(1).eq(1) & (fgl.timeout_team.shift(1) == fgl.defteam))
late_fg = fgl[(fgl.play_type == "field_goal") & (fgl.qtr == 4) & (fgl.game_seconds_remaining <= 120)].copy()
late_fg["iced"] = np.where(prev_def_to.loc[late_fg.index], "Iced (defense's timeout just before)",
                           "Not iced")
late_fg["made"] = (late_fg.field_goal_result == "made").astype(float)
late_fg["expected"] = p_fg(late_fg.kick_distance)
ice = late_fg.groupby("iced").agg(kicks=("made", "size"), made=("made", "mean"),
                                expected=("expected", "mean"), distance=("kick_distance", "mean"))
ice["made - expected"] = ice.made - ice.expected
ice_se = np.sqrt(0.2 * 0.8 / ice.loc[ice.index.str.startswith("Iced"), "kicks"].iloc[0])
md_table(ice.assign(made=100 * ice.made, expected=100 * ice.expected,
                    **{"made - expected": 100 * ice["made - expected"]})
         .rename(columns={"made": "made (%)", "expected": "expected (%)",
                           "made - expected": "made - expected (pts)"}).reset_index()
         .rename(columns={"iced": "last-2-minute field goals, 2016-2025"}),
         widths=[24, 6, 7, 8, 7, 9])
```

last-2-minute field goals, 2016-2025	kicks	made (%)	expected (%)	distance	made - expected (pts)
Iced (defense's timeout just before)	161	76.4	82.5	42.3	-6.1
Not iced	559	79.6	83.1	41.4	-3.5

Iced kicks were made about 6 points less often than their distance predicts. But un-iced kicks in the same last two minutes also fell short, by about 3 points (pressure, weather and bad spots are part of every late kick), so icing's own effect is the difference, a couple of points, and with fewer than 200 iced kicks the uncertainty (a standard error of about 3 points) is larger than that. Earlier studies reached the same verdict: no reliable evidence that icing works, and a real cost, because the timeout might have been worth more saved for the next possession.¹⁶ This is the pattern of the whole chapter in miniature: a tradition that feels powerful, a data set that can't confirm it, and a model that prices what the tradition costs.

¹⁶Tobias J. Moskowitz and L. Jon Wertheim, *Scorecasting* (New York: Crown Archetype, 2011), chapter on icing the kicker (no measurable effect in NFL data).

Watch for it on Sunday

💡 Watch for it

Be the bot. Make every fourth-down call live, before the offense decides, then check it against a model afterwards.

- **Before the snap on fourth down, find the break-even rate in your head.** Short yardage between the 20s: you need to convert about half the time, and league-average offenses convert fourth-and-1 about two times in three and fourth-and-2 a bit under 60%. Near the goal line the bar drops into the 20s or 30s. In the opponent's 30-40 (the go zone), almost anything up to fourth-and-4 or 5 says go.
- **Then adjust for the score and clock.** Leading late: kicks gain value. Trailing late: kicks lose it, especially a field goal that doesn't tie.
- **Then adjust for the teams.** A great offense or a weak defense raises *this* play's conversion chance; a strong kicker makes the 50-yarder more attractive. This is the part the bot can't do.
- **Make your call, and say how close it is.** "Go, easily" or "toss-up, lean go." The size matters more than the sign.
- **After a touchdown when down 14,** say "go for two" before the extra-point unit runs on, and watch whether the coach agrees (about half do now).
- **Watch the broadcast win-probability bar on the big plays** and estimate the WPA: which play of the drive moved it most, and was it a scoring play or a high-leverage third down?
- **Afterwards,** look the decision up: the nfl4th calculator, a team's analytics account, or the code in this chapter (`decide(lead, yardline_100, yards_to_go, minutes_left)`). Log your calls in your prediction log with the model's gain; Film Study and Charting shows how to score your calibration.

Predict the play

The pictures use the course's position letters: on offense X, Z and H are wide receivers, Y and U tight ends, F the fullback and T or R the running back; on defense E is an end, T a tackle, N a nose tackle, S, M and W the strong-side, middle and weak-side linebackers, C a cornerback, \$ the nickel back, and FS and SS the safeties.

```
from gridiron import Play, offense, defense

def situation(off, dfn, los, to_go, title, lateral=(-16, 16), window=None, width_in=5.6, notes=()):
    """A pre-snap picture with the line to gain in yellow."""
    p = Play(off, dfn, los=los, to_go=to_go, title=title)
    for text, at in notes:
        p.note(text, at=at)
    p.draw(lateral=lateral, window=window, width_in=width_in)
    plt.show()
```

💡 Predict the play

Fourth-and-1 at your own 34, first quarter, 0-0. The offense stays on the field in a two-back I-formation; the defense brings its base 4-3 with the strong safety down, eight in the box. Should the offense go for it or punt? What is the break-even conversion rate, and what did NFL teams actually do here a decade ago and in 2025?

```
off = offense("i_form")
dfn = [q.moved(d=1.5) if q.pos == "CB" else q          # short yardage: corners press
      for q in defense("4-3_under", "single_high", off)]
situation(off, dfn, los=44, to_go=1, title="4th & 1, own 34, 1st quarter, 0-0",
          lateral=(-17, 17), window=(-9, 15))
```

4th & 1, own 34, 1st quarter, 0-0

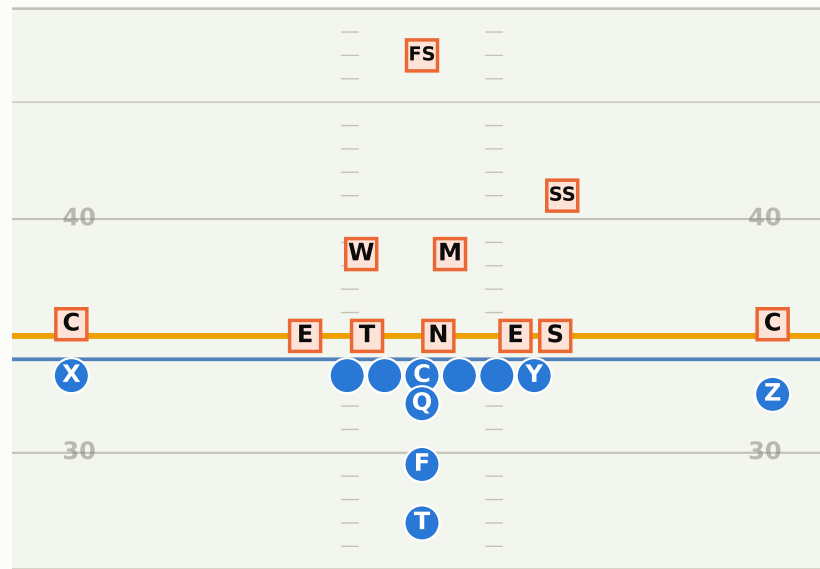


Figure 13: Fourth-and-1 at the offense's own 34, first quarter, tied: 21 personnel in an I-formation (fullback and tailback behind the quarterback, one tight end) against a base 4-3 Under front with the strong safety rolled into the box and both corners pressed. The yellow line to gain is one yard away.

Answer 1 is at the end of the chapter. Try it first.

💡 Predict the play

Fourth-and-3 at the opponent's 8, fourth quarter, 6:30 left. You trail 21-17. The offense lines up in a three-receiver set, and the defense is in nickel with everyone within about 8 yards of the line, as it must be this close to the end zone. Kick the 26-yard field goal to make it 21-20, or go for it? How does the answer differ from the same fourth down in a tied first quarter?

```
off = [q.moved(w=-17.5) if q.key == "X" else q for q in offense("gun_trips")]
dfn = []
for q in defense("nickel", "two_high", off):
    if q.key in ("FS", "SS"):
        q = q.moved(d=8.0) # end line is 18 yards from the LOS
    elif q.pos == "CB":
        q = q.moved(d=4.0)
    dfn.append(q)
situation(off, dfn, los=102, to_go=3, title="4th & 3 at the 8, down 21-17, 6:30 left",
          lateral=(-20, 20), window=(-8, 18))
```

4th & 3 at the 8, down 21-17, 6:30 left

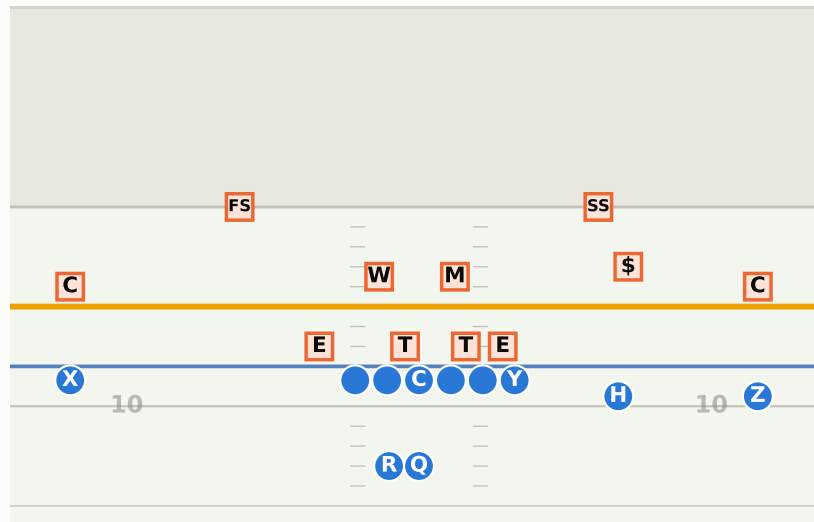


Figure 14: Fourth-and-3 at the opponent's 8, trailing by 4 with 6:30 left: 11 personnel in a 3x1 shotgun trips set against a nickel defense with both safeties near the goal line and the end zone behind them. The yellow line to gain is at the 5.

Answer 2 is at the end of the chapter. Try it first.

💡 Predict the play

You trailed 22-8, and with 9:00 left in the fourth quarter you have just scored a touchdown: it is 22-14. The offense stays on the field at the 2. Kick the extra point (down 7) or go for two (down 6 if it works, still down 8 if it doesn't)? What do teams do now?

```
off = offense("gun_2x2")
dfn = []
for q in defense("nickel", "two_high", off):
    if q.key in ("FS", "SS"):
        q = q.moved(d=6.5, w=np.sign(q.w) * 6.0)      # inside the slots, deep in the end zone
    elif q.pos in ("CB", "NB"):
        q = q.moved(d=2.0)                            # tight on the receivers
    dfn.append(q)
situation(off, dfn, los=108, to_go=2, title="Down 8 after the TD, 9:00 left: kick or go?",
          lateral=(-17, 17), window=(-8, 12))
```

Down 8 after the TD, 9:00 left: kick or go?

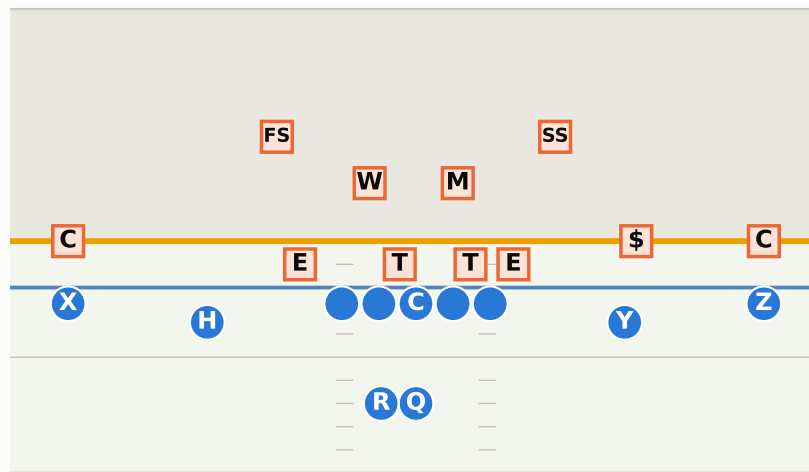


Figure 15: The try after a touchdown, snapped from the 2: an 11-personnel shotgun 2x2 set against a nickel defense squeezed into the end zone (the end line is 12 yards behind the line of scrimmage). The yellow line is the goal line.

Answer 3 is at the end of the chapter. Try it first.

Takeaways

- **Win probability is a model of the rest of the game.** A random walk of points, scaled by $\sqrt{(\text{time left})}$ and started from the lead plus the possession's expected points, matches nflverse's machine-learning model on 2025 data; the betting line adds what both lack, which team is better. Check a model by its calibration, not by one game: 99% means one collapse in a hundred, and Super Bowl LI was one.
- **WPA scores plays by how much they changed the chance of winning**, and leverage says how much a situation lets a play matter. A tenth of plays carry about two-fifths of all win-probability movement; most of the game is setting up the few snaps that decide it.
- **A fourth-down decision is a probability tree.** Conversion probability times the win probability of each outcome, against the field goal and the punt. The break-even conversion rate is the one number to carry: go if you believe the play converts more often than that.
- **The map says go on short yardage almost everywhere**, go in the opponent's 30-40, and punt on long yardage in your own half; the score and clock then shift it: kicks gain value when leading late and lose it when trailing.

- **Respect the model's limits.** Simple models break in the last five minutes, where scoring is lumpy and the clock is controllable. Every model assumes average teams, and models disagree by a point or more on close calls. Read the size of the gain, not just its sign.
- **Coaches moved toward the math, partly.** On clear-go fourth downs the go rate doubled from 2016 to 2025 but stayed under half; the down-14 two-point decision went from never to about half of fourth-quarter tries. Detroit followed the model more often than any other team.

Connections

Builds on: Expected Points, EPA, and Success Rate (the EP values that become the “possession” term in the WP model), The Clock, the Two-Minute Drill, and Fourth Down (the decisions, heuristics and go-rate history), and Working with nflverse Data in Python. The history of the 2009 fourth-and-2 is in Dynasties and Counterpunches, and the aggression era in The Modern Era.

Sets up:

- Tendencies: PROE, Personnel, Motion, and Coverage: the neutral-situation filter and “over expected” metrics, which use win probability to decide which plays count.
- Film Study and Charting: calibration and the Brier score applied to your own predictions, including your fourth-down calls.
- Case Study: The Lions and Case Study: The Eagles: the two franchises that made short-yardage fourth downs part of their identity.

If you want to go deeper: Ben Baldwin’s nfl4th documentation and his nflfastR model write-ups on Open Source Football;¹⁷ Hal Stern’s two papers on the normal and Brownian-motion models; David Romer’s “Do Firms Maximize?”; and Brill, Yurko and Wyner’s “Analytics, have some humility” for the case that the models are less certain than their graphics. For the clock and score logic behind two-point charts, the FiveThirtyEight piece on Dick Vermeil’s chart cited in 10-04 is a good start.

Answers to Predict the play

Answer 1.

```
dr1 = decide(0, 66, 1, 52).iloc[0]
zone = reg[(reg.down == 4) & reg.play_type.isin(["pass", "run", "punt", "field_goal"])
           & (reg.ydstogo == 1) & reg.yardline_100.between(60, 70) & (reg.qtr <= 3)
           & reg.wp.between(0.3, 0.7)]
went = zone.groupby("season").play_type.apply(lambda s: s.isin(["pass", "run"]).mean())
print(f"Go {dr1.go:.1%} vs punt {dr1.punt:.1%}: gain {100 * dr1.gain:+.1f} pts; "
      f"break-even {dr1.breakeven:.0%}; P(convert) {dr1.p_conv:.0%}")
print(f"Teams went for it on 4th & 1 at their own 30-40 (Q1-3, close games): "
      f"{went.loc[2016]:.0%} in 2016, {went.loc[2025]:.0%} in 2025 ({len(zone)} fourth downs)")
```

```
Go 50.6% vs punt 47.6%: gain +3.0 pts; break-even 48%; P(convert) 67%
Teams went for it on 4th & 1 at their own 30-40 (Q1-3, close games): 4% in 2016, 61% in 2025 (322 fourth downs)
```

Go. The break-even rate is just under 50%, and a league-average offense converts fourth-and-1 about two times in three; with a quarterback sneak or a tush push available, more. Failure hands the opponent the ball at your 34, which hurts, but a punt from here only moves the ball about 40 yards, and the conversion keeps a drive alive with three more downs. The gain of about three points of win probability is one of the largest routine edges on the fourth-down map.

And the behaviour change is dramatic: in 2016, teams in close games went for it on fourth-and-1 at their own 30 to 40 about 4% of the time; in 2025, about 61%. The eight-man box doesn’t change the answer much. It makes the run a little harder and the play-action pass a little easier, which is a reason to pick the play carefully, not a reason to punt.

Answer 2.

¹⁷nfl4th documentation and research notes: <https://www.nfl4th.com/articles/4th-down-research.html>; nflfastR model documentation: <https://www.nflfastR.com/>.

```
late_r = decide(-4, 8, 3, 6.5).iloc[0]
early_r = decide(0, 8, 3, 55).iloc[0]
print(f"Down 4, 6:30 left: go {late_r.go:.1%}, FG {late_r.fg:.1%} -> gain {100 * late_r.gain:+.1f} "
      f"pts, break-even {late_r.breakeven:.0%}")
print(f"Tied, Q1: go {early_r.go:.1%}, FG {early_r.fg:.1%} -> gain {100 * early_r.gain:+.1f} "
      f"pts, break-even {early_r.breakeven:.0%}")
```

```
Down 4, 6:30 left: go 44.7%, FG 33.1% -> gain +11.6 pts, break-even 22%
Tied, Q1: go 58.2%, FG 53.8% -> gain +4.4 pts, break-even 25%
```

Go, clearly. The field goal is nearly automatic, but it leaves you trailing 21-20 and kicking off with six minutes left, still needing more points. A conversion leaves first-and-goal inside the 5, where a touchdown is likely and puts you ahead; a failure leaves the opponent backed up at its own 8, so even the bad branch keeps you in the game. The break-even conversion rate is around 22%, far below the roughly 50% that teams convert on fourth-and-3.

The same fourth down in a tied first quarter is still a go, but by about 4 points instead of 12: early, three points are worth about their face value, and a near-certain field goal is a decent result. Late and behind, the field goal's three points are worth much less than three, because they don't change what you still need. That difference, between points and wins, is the whole reason this chapter exists.

Answer 3.

Go for two. This is the down-14 problem: you were down 14 (22-8), you scored, and you are down 8 before the try. Using the tree from this chapter with the league's 2016-2025 rates, going for two now wins about 59% of the games in which you score the next touchdown and stop the opponent, against 46% for kicking, and it is better whenever your two-point play succeeds more than about 35% of the time. If the try fails, you know with nine minutes left that you need a touchdown and a two-pointer, and you can manage fourth downs and timeouts accordingly.

What do teams do? In 2016 and 2017 nobody in this spot went for two. In 2025, about 53% of fourth-quarter tries at down 8 were two-point attempts: the fastest-moving decision in this chapter, and a sign that when a decision is a single clean tree, coaches adopt the math quickly.